

Advanced thread scheduling

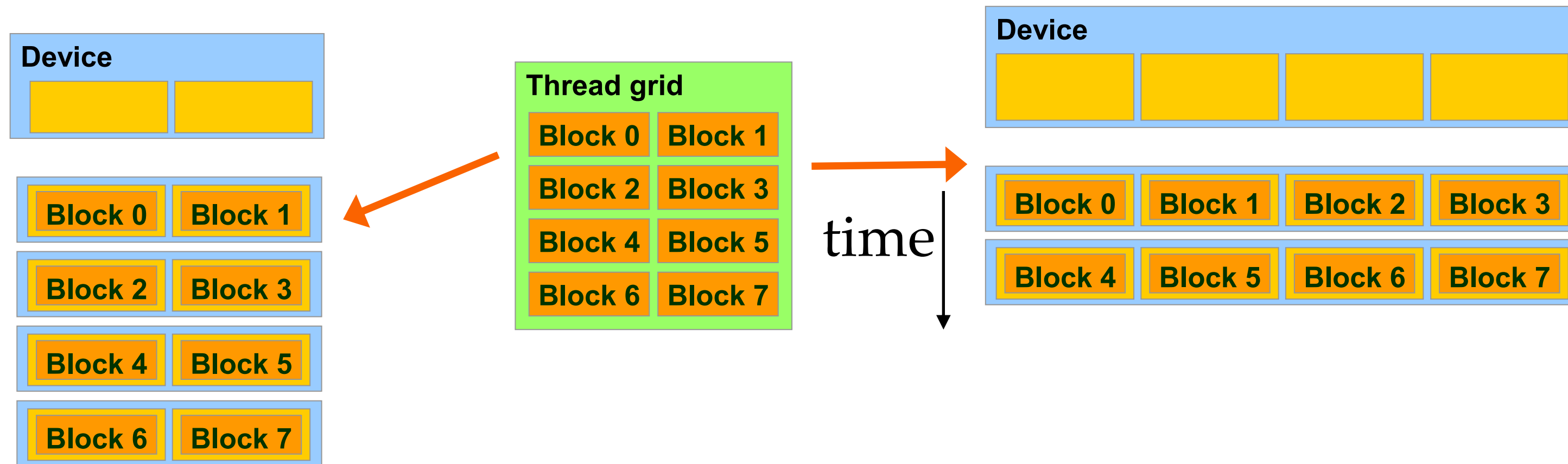
Material from The GPU Teaching Kit .

The GPU Teaching Kit is licensed by NVIDIA and the University of Illinois under the [Creative Commons Attribution-NonCommercial 4.0 International License.](#)

Objective

- To learn how a CUDA kernel utilizes hardware execution resources
 - Assigning thread blocks to execution resources
 - Capacity constraints of execution resources
 - Zero-overhead thread scheduling

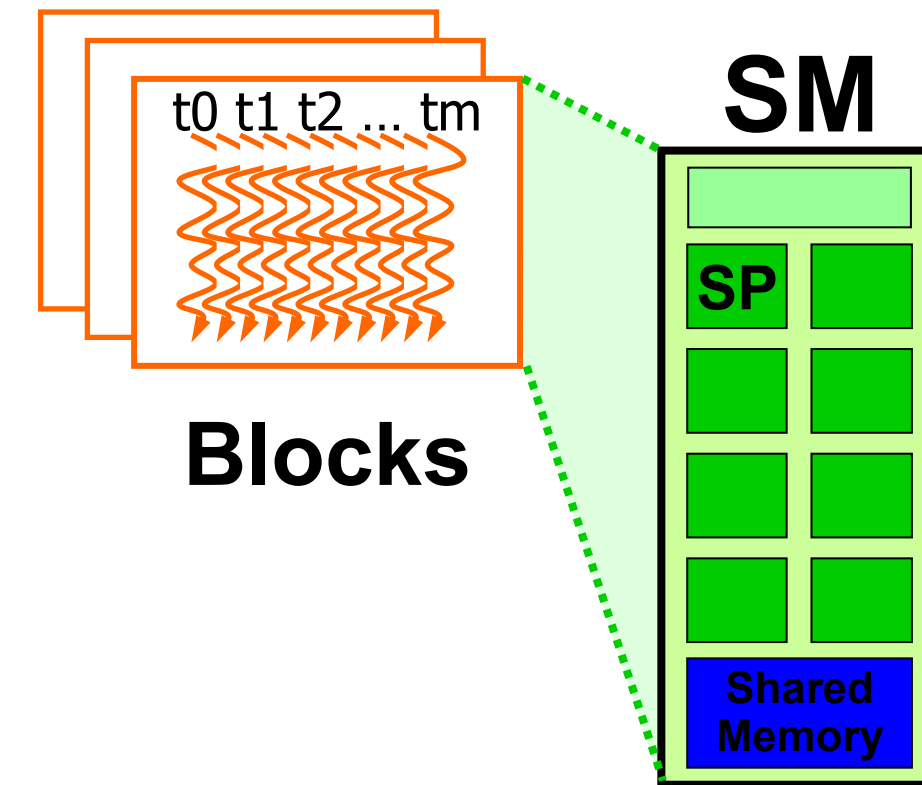
Transparent Scalability



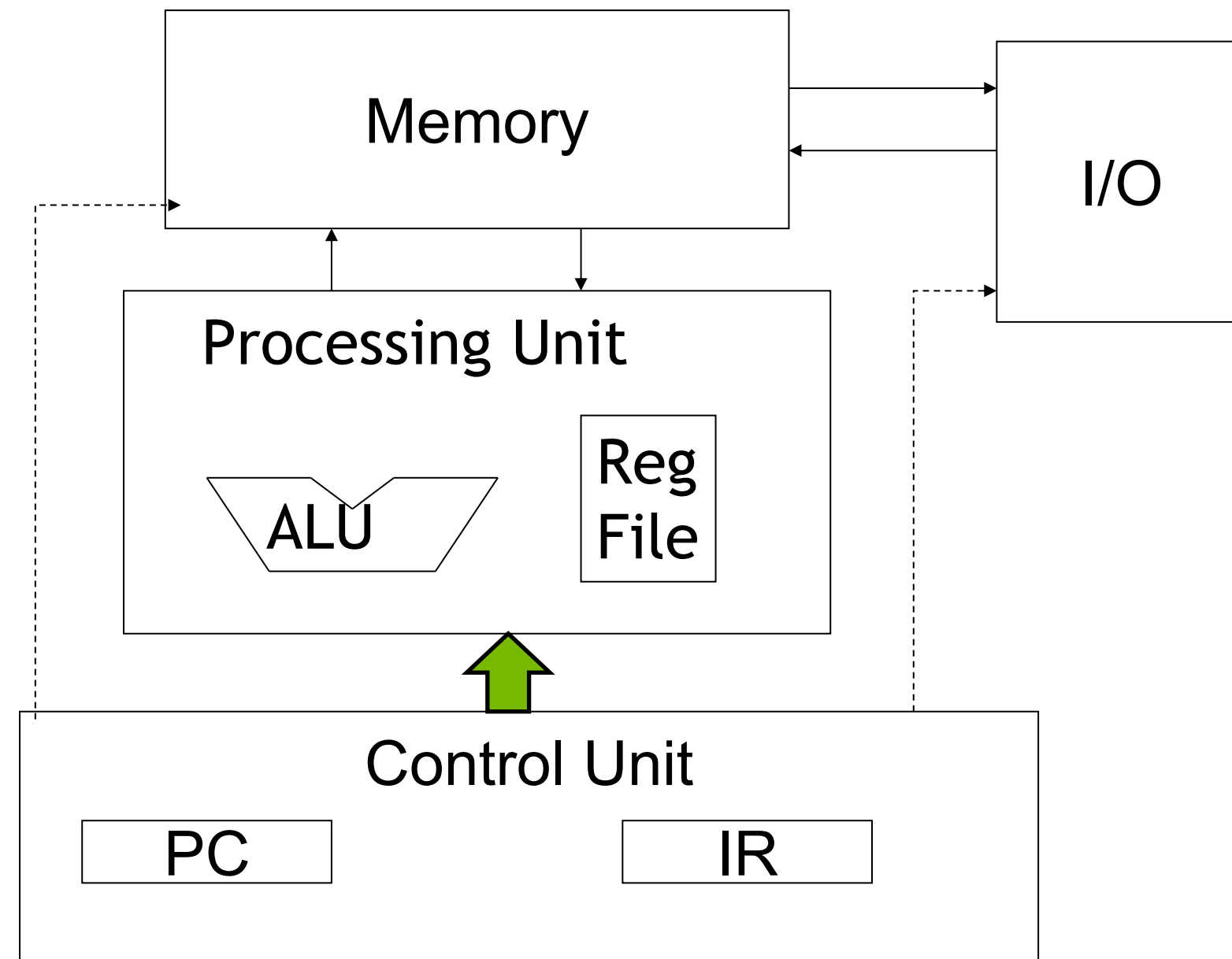
- Each block can execute in any order relative to others.
- Hardware is free to assign blocks to any processor at any time
 - A kernel scales to any number of parallel processors

Example: Executing Thread Blocks

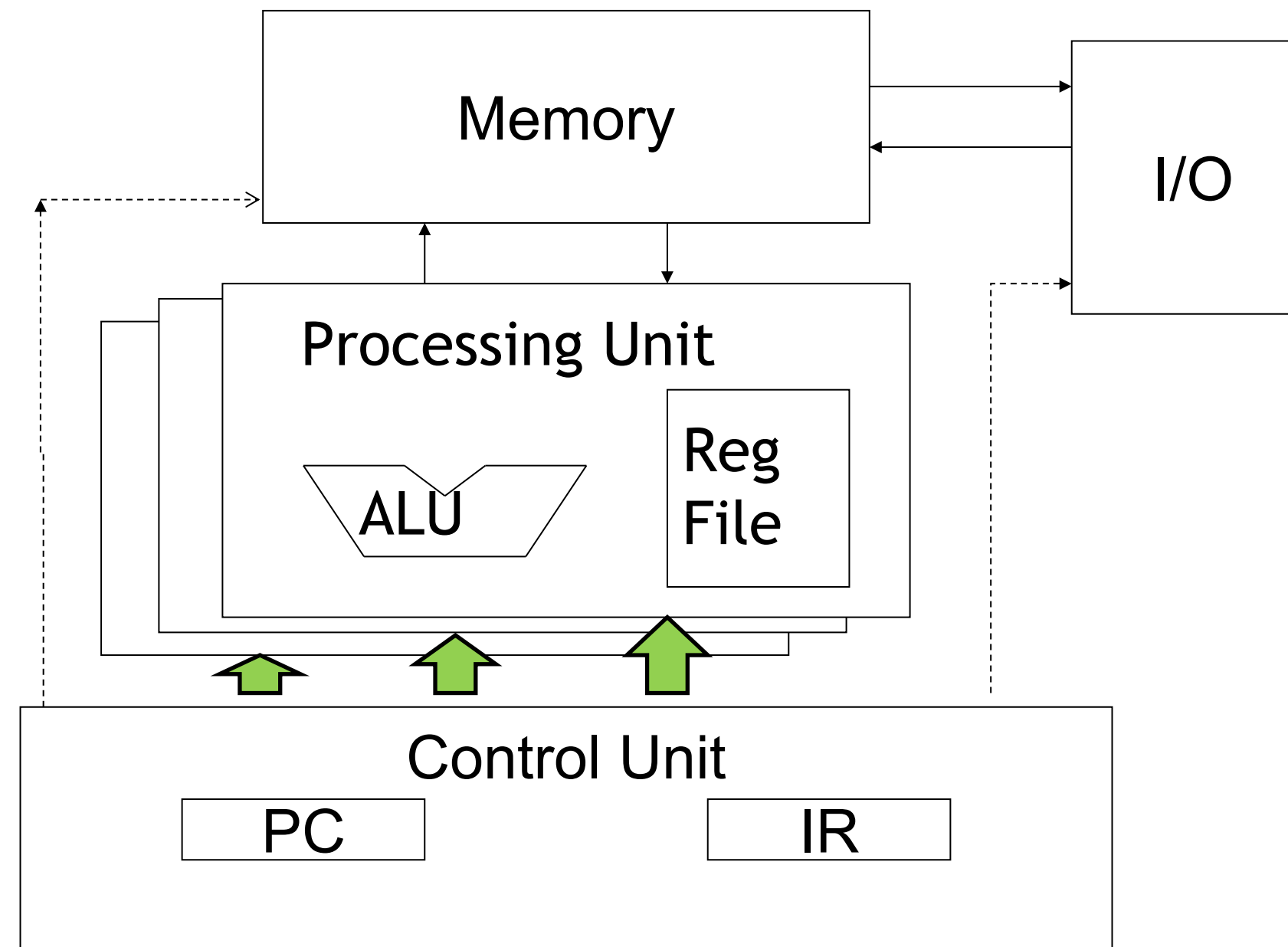
- Threads are assigned to **Streaming Multiprocessors (SM)** in block granularity
 - Up to 32 blocks to each SM as resource allows
 - Volta SM can take up to **2048** threads
 - Could be 256 (threads/block) * 8 blocks
 - Or 512 (threads/block) * 4 blocks, etc.
- SM maintains thread/block idx #s
- SM manages/schedules thread execution



The Von-Neumann Model



The Von-Neumann Model with SIMD units



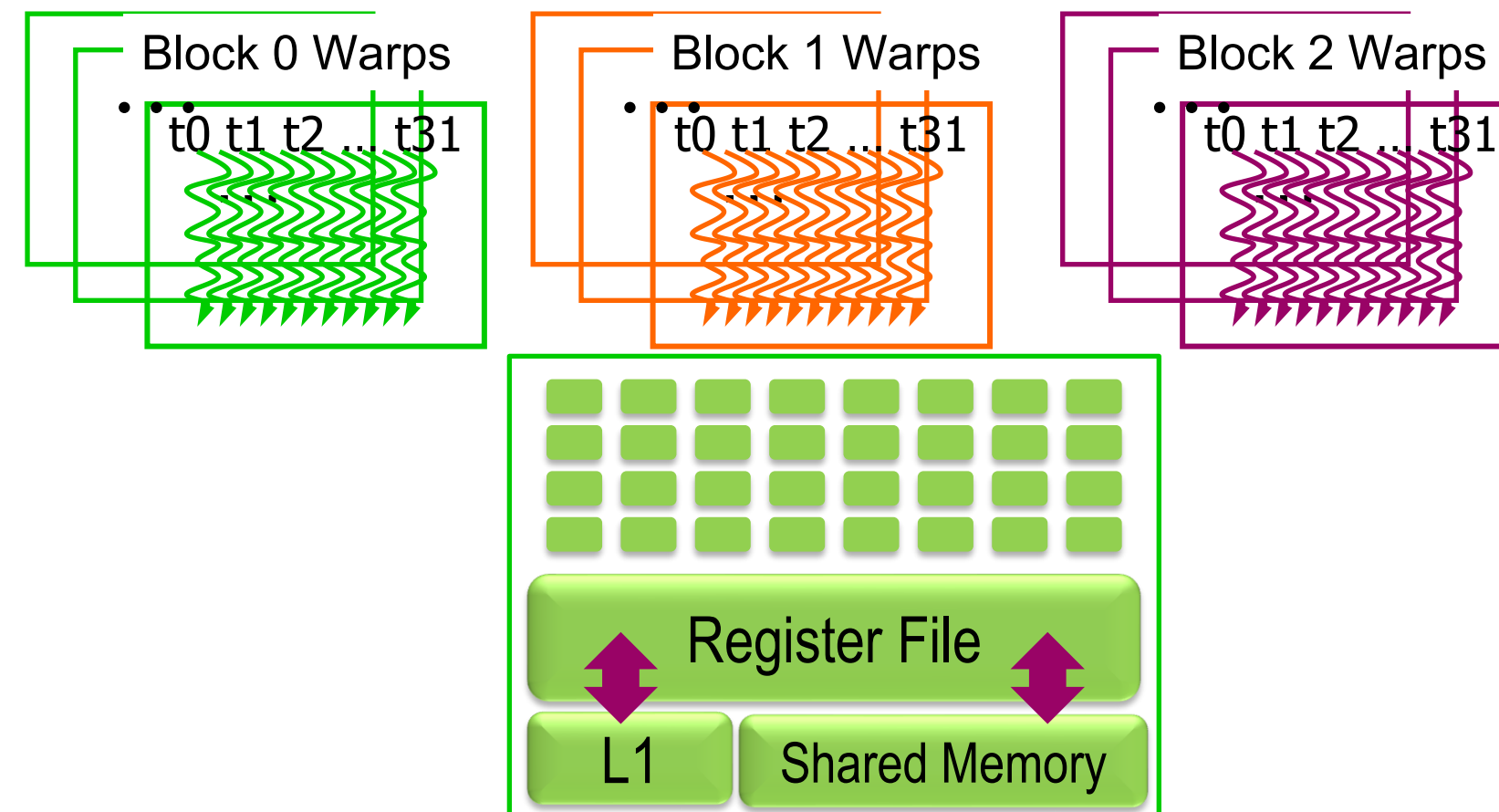
Single Instruction Multiple Data
(SIMD)

Warps as Scheduling Units

- Each Block is executed as 32-thread Warps
 - An implementation decision, not part of the CUDA programming model
 - Warps are scheduling units in SM
 - Threads in a warp execute in SIMD
 - Future GPUs may have different number of threads in each warp

Warp Example

- If 3 blocks are assigned to an SM and each block has 256 threads, how many Warps are there in an SM?
 - Each Block is divided into $256/32 = 8$ Warps
 - There are $8 * 3 = 24$ Warps



Example: Thread Scheduling (Cont.)

- SM implements zero-overhead warp scheduling
 - Warps whose next instruction has its operands ready for consumption are eligible for execution
 - Eligible Warps are selected for execution based on a prioritized scheduling policy
 - All threads in a warp execute the same instruction when selected

Block Granularity Considerations

- For Matrix Multiplication using multiple blocks, should each block have 4X4, 8X8 or 30X30 threads for Volta?
- For 4X4, we have 16 threads per Block. Each SM can take up to 2048 threads, which translates to 128 Blocks. However, each SM can only take up to 32 Blocks, so only 512 threads will go into each SM!
- For 8X8, we have 64 threads per Block. Since each SM can take up to 2048 threads, it can take up to 32 Blocks and achieve full capacity unless other resource considerations overrule.
- For 30X30, we would have 900 threads per Block. Only two blocks could fit into an SM for Volta, so only $1800/2048$ of the SM thread capacity would be utilized.

Comparison of NVIDIA Data Center GPUs

GPU Features	NVIDIA Pascal P100	NVIDIA Volta V100	NVIDIA A100
GPU Codename	GP100	GV100	GA100
GPU Architecture	NVIDIA Pascal	NVIDIA Volta	NVIDIA Ampere
GPU Board Form Factor	SXM	SXM2	SXM4
SMs	56	80	108
TPCs	28	40	54
FP32 Cores/ SM	64	64	64
FP32 Cores/ GPU	3584	5120	6912
FP64 Cores/ SM (excl. Tensor)	32	32	32
FP64 Cores/ GPU (excl. Tensor)	1792	2560	3456
INT32 Cores / SM	NA	64	64
INT32 Cores / GPU	NA	5120	6912
Tensor Cores / SM	NA	8	4 ²
Tensor Cores / GPU	NA	640	432
GPU Boost Clock	1480 MHz	1530 MHz	1410 MHz
Memory Interface	4096-bit HBM2	4096-bit HBM2	5120-bit HBM2
Memory Size	16 GB	32 GB / 16 GB	40 GB
Memory Data Rate	703 MHz DDR	877.5 MHz DDR	1215 MHz DDR
Memory Bandwidth	720 GB/sec	900 GB/sec	1555 GB/sec
L2 Cache Size	4096 KB	6144 KB	40960 KB
Shared Memory Size / SM	64 KB	Configurable up to 96 KB	Configurable up to 164 KB
Register File Size / SM	256 KB	256 KB	256 KB
Register File Size / GPU	14336 KB	20480 KB	27648 KB
TDP	300 Watts	300 Watts	400 Watts
Transistors	15.3 billion	21.1 billion	54.2 billion
GPU Die Size	610 mm ²	815 mm ²	826 mm ²
TSMC Manufacturing Process	16 nm FinFET+	12 nm FFN	7 nm N7

Comparison of Compute Capability: GP100 vs GV100 vs GA100

GPU features	NVIDIA Pascal P100	NVIDIA Volta V100	NVIDIA A100
GPU codename	GP100	GV100	GA100
GPU architecture	NVIDIA Pascal	NVIDIA Volta	NVIDIA Ampere
Compute capability	6.0	7.0	8.0
Threads / warp	32	32	32
Max warps / SM	64	64	64
Max threads / SM	2048	2048	2048
Max thread blocks / SM	32	32	32
Max 32-bit registers / SM	65536	65536	65536
Max registers / block	65536	65536	65536
Max registers / thread	255	255	255
Max thread block size	1024	1024	1024
FP32 cores / SM	64	64	64
Ratio of SM registers to FP32 cores	1024	1024	1024
Shared Memory Size / SM	64 KB	Configurable up to 96 KB	Configurable up to 164 KB

Memory and Data Locality in depth

Material from The GPU Teaching Kit .

The GPU Teaching Kit is licensed by NVIDIA and the University of Illinois under the [Creative Commons Attribution-NonCommercial 4.0 International License.](#)

Objective

- To learn to effectively use the CUDA memory types in a parallel program
 - Importance of memory access efficiency
 - Registers, shared memory, global memory
 - Scope and lifetime

TL;DR: how to use shared memory to compensate for the limited performance of global memory bandwidth

Review: Image Blur Kernel.

```
// Get the average of the surrounding 2xBLUR_SIZE x 2xBLUR_SIZE box
for(int blurRow = -BLUR_SIZE; blurRow < BLUR_SIZE+1; ++blurRow) {
    for(int blurCol = -BLUR_SIZE; blurCol < BLUR_SIZE+1; ++blurCol) {

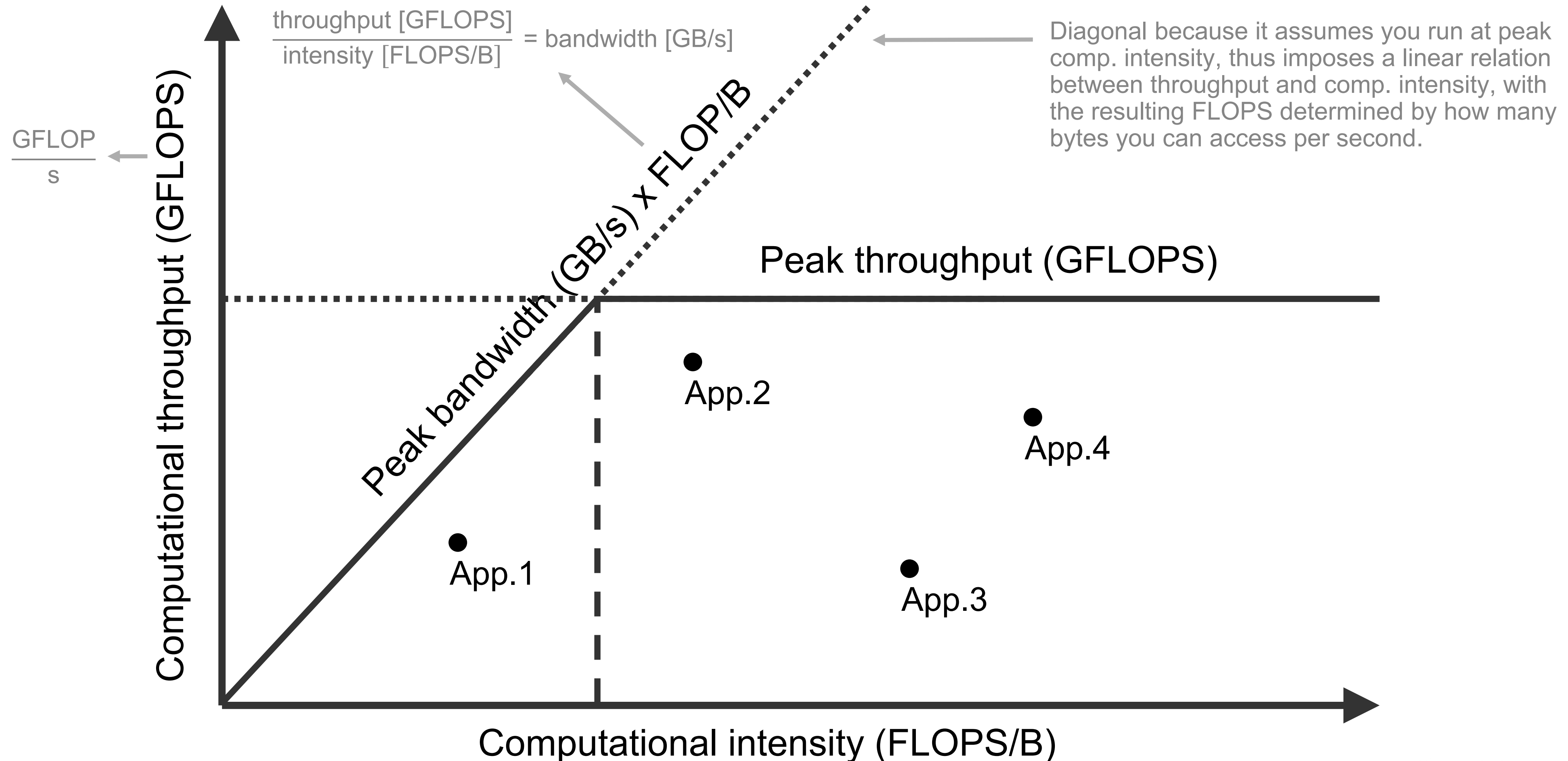
        int curRow = Row + blurRow;
        int curCol = Col + blurCol;
        // Verify we have a valid image pixel
        if(curRow > -1 && curRow < h && curCol > -1 && curCol < w) {
            ➡ pixVal += in[curRow * w + curCol];
            pixels++; // Keep track of number of pixels in the accumulated total
        }
    }
}

// Write our new pixel value out
out[Row * w + Col] = (unsigned char)(pixVal / pixels);
```

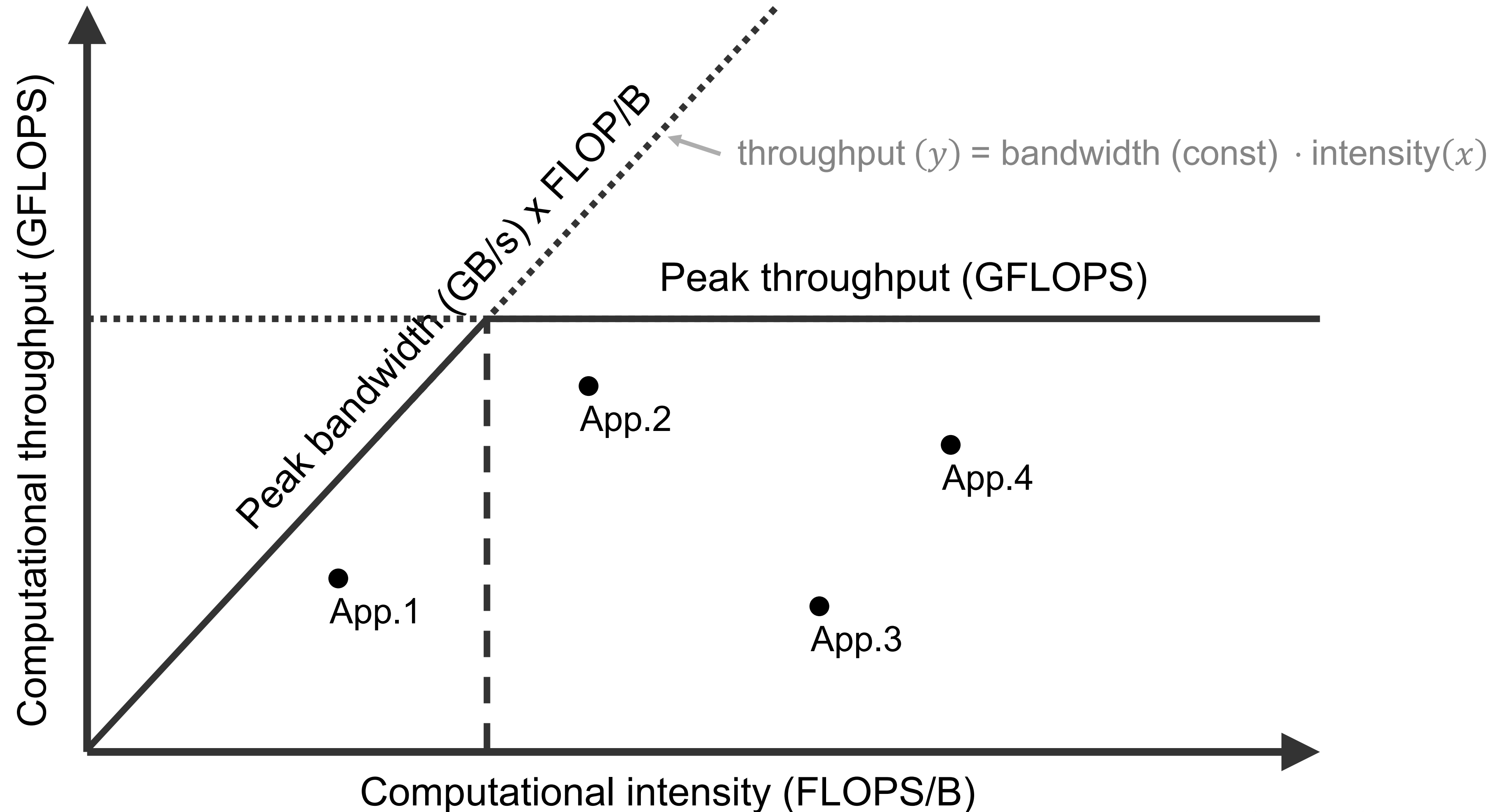
How about performance on a GPU

- Compute to global memory access ratio (arithmetic intensity)
 - # FLOPs performed for each byte access from global memory within a program region
- All threads access global memory for their input elements
 - One memory accesses (4 bytes) per floating-point addition
 - **(4B/s of memory bandwidth)/FLOPS**
 - $\frac{1}{4}$ FLOPS/B arithmetic intensity
- Assume a GPU with
 - Peak floating-point rate 1,600 GFLOPS with 600 GB/s DRAM bandwidth
 - $4 \times 1,600 = 6,400$ GB/s required to achieve peak FLOPS rating
 - The 600 GB/s memory bandwidth limits the execution at 150 GFLOPS
- This limits the execution rate to 9.3% ($150/1600$) of the peak floating-point execution rate of the device!
- Need to drastically cut down memory accesses to get close to the 1,600 GFLOPS

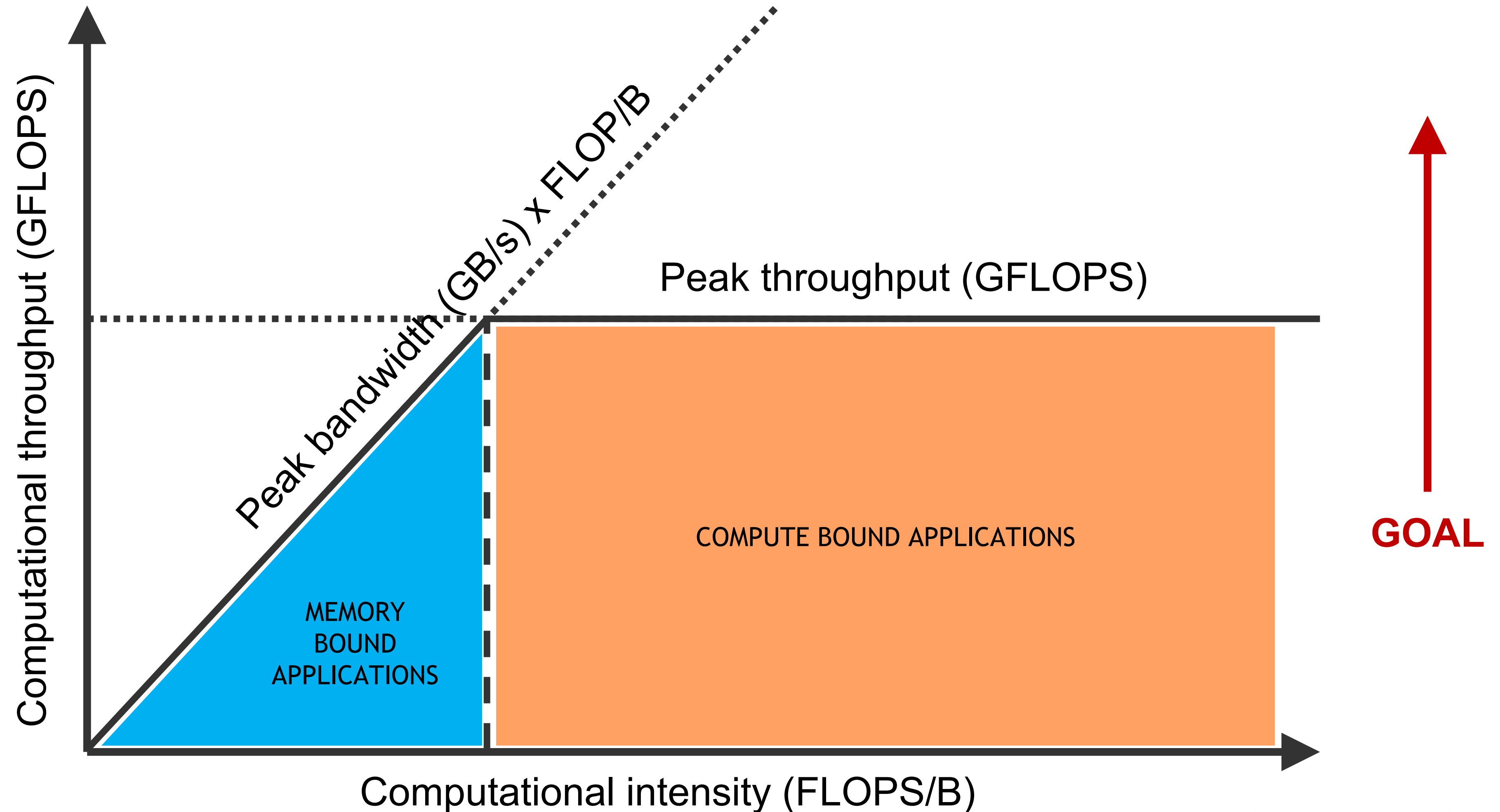
The Roofline Model: FLOP per Byte VS FLOP per second



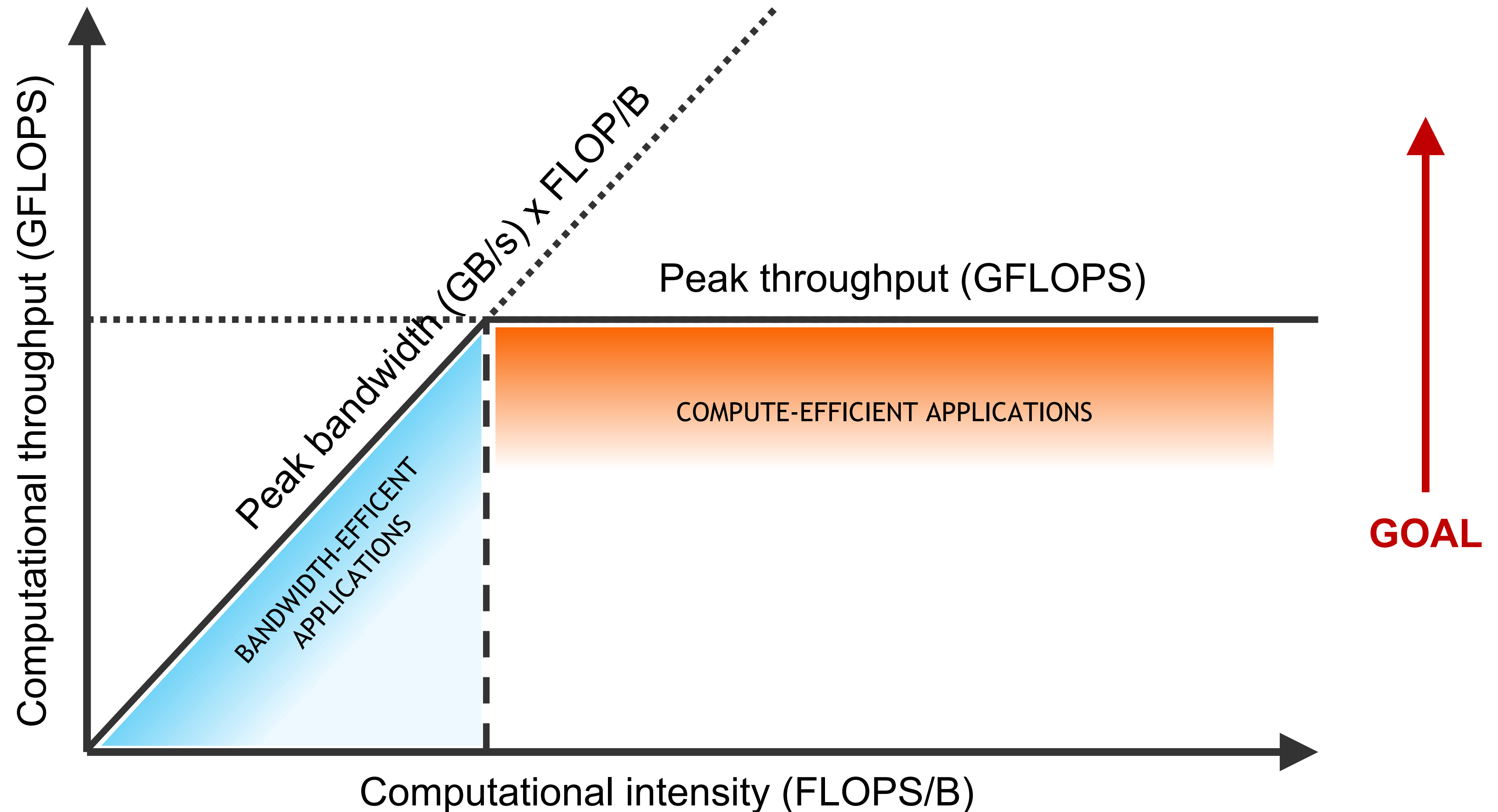
The Roofline Model: FLOP per Byte VS FLOP per second



The Roofline Model: Application Characterization



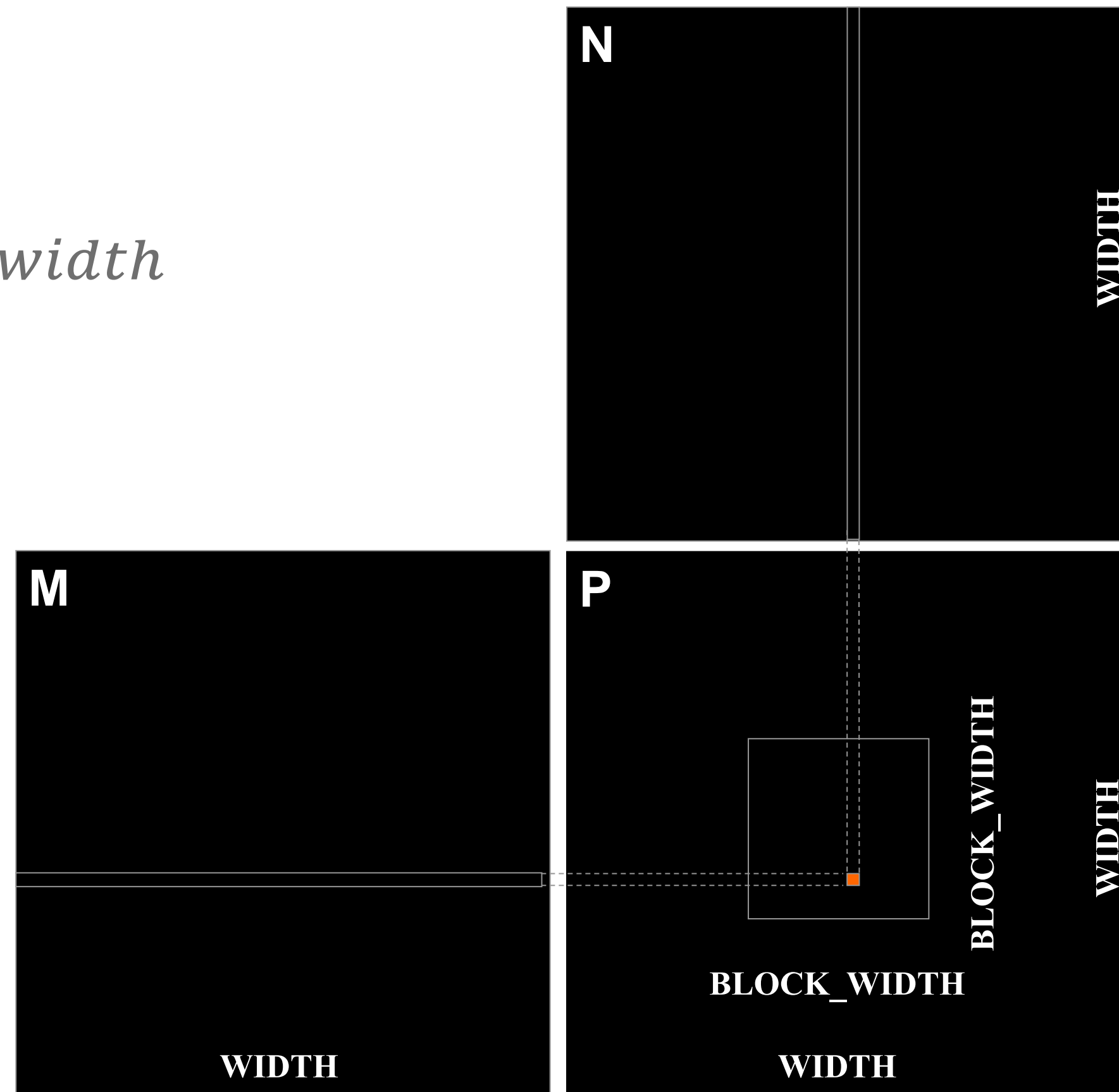
The Roofline Model: Aiming for Efficiency



Example – Matrix Multiplication

$$P = M \otimes N$$

$$M, N, P \in \mathbb{M}^{width \times width}$$



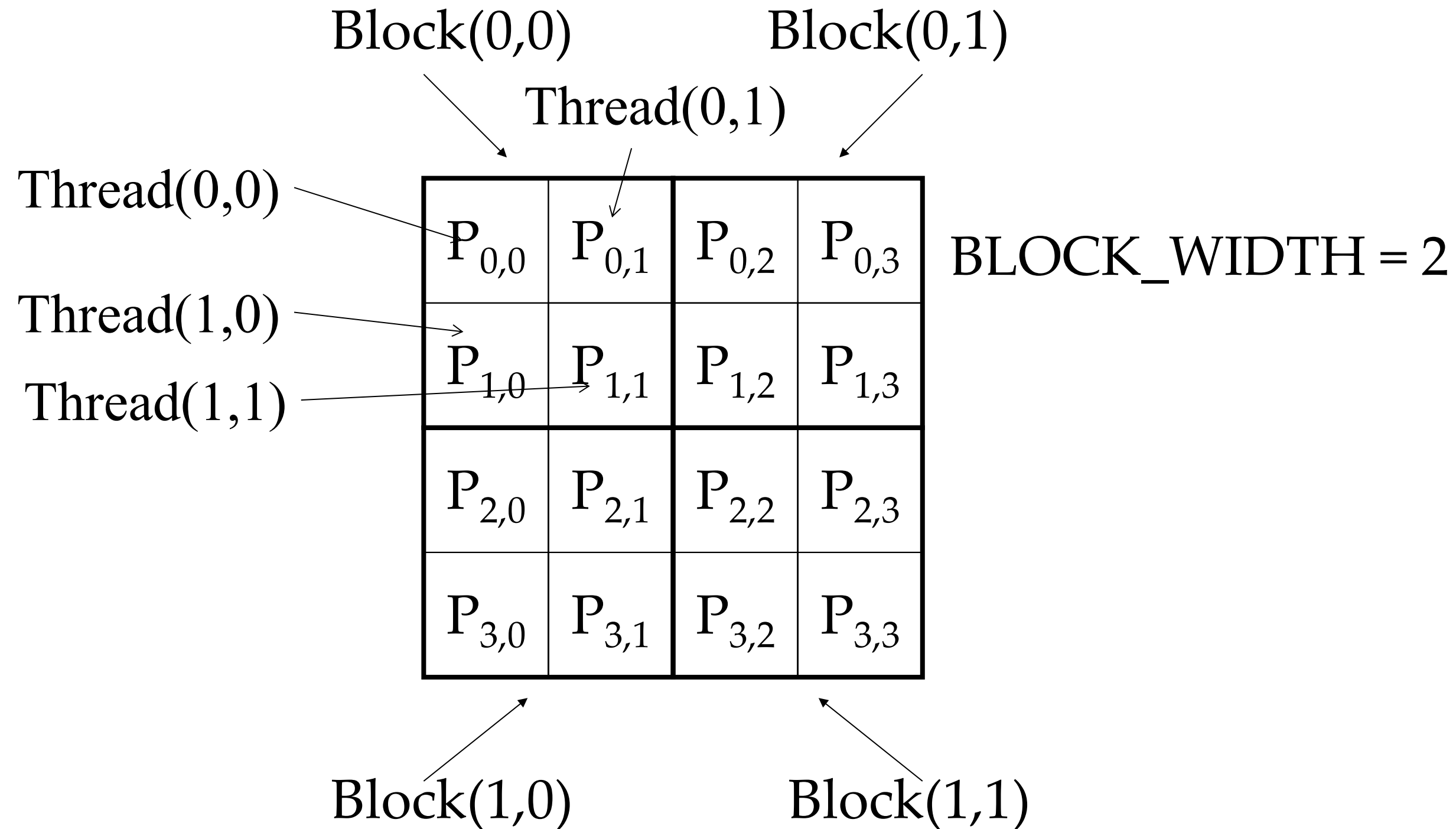
A Basic Matrix Multiplication

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {  
  
    // Calculate the row index of the P element and M  
    int Row = blockIdx.y*blockDim.y+threadIdx.y;  
  
    // Calculate the column index of P and N  
    int Col = blockIdx.x*blockDim.x+threadIdx.x;  
  
    if ((Row < Width) && (Col < Width)) {  
        float Pvalue = 0;  
        // each thread computes one element of the block sub-matrix  
        for (int k = 0; k < Width; ++k) {  
            Pvalue += M[Row*Width+k]*N[k*Width+Col];  
        }  
        P[Row*Width+Col] = Pvalue;  
    }  
}
```

Example – Matrix Multiplication

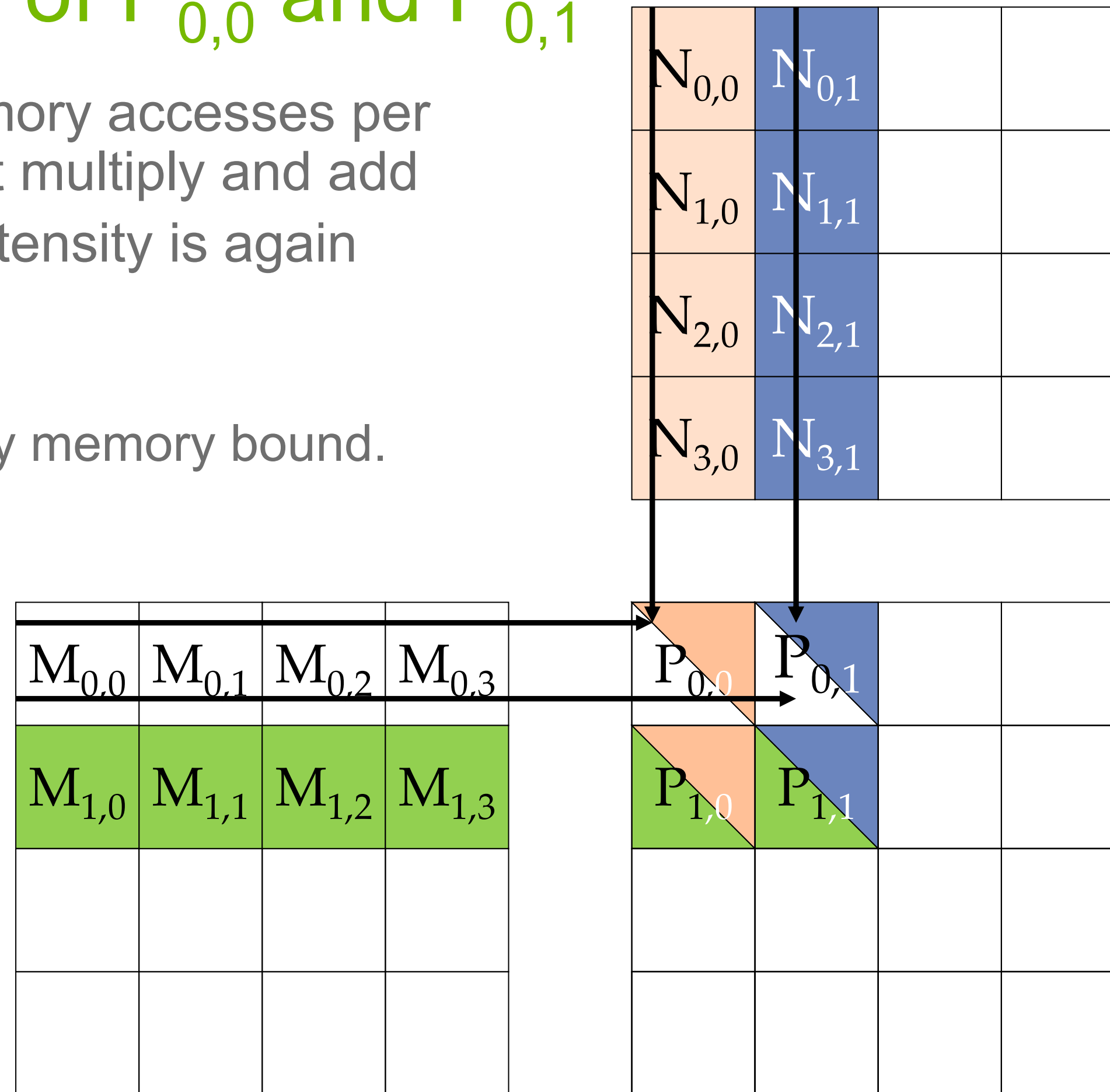
```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width) {  
  
    // Calculate the row index of the P element and M  
    int Row = blockIdx.y*blockDim.y+threadIdx.y;  
  
    // Calculate the column index of P and N  
    int Col = blockIdx.x*blockDim.x+threadIdx.x;  
  
    if ((Row < Width) && (Col < Width)) {  
        float Pvalue = 0;  
        // each thread computes one element of the block sub-matrix  
        for (int k = 0; k < Width; ++k) {  
            Pvalue += M[Row*Width+k]*N[k*Width+Col];  
        }  
        P[Row*Width+Col] = Pvalue;  
    }  
}
```

A Toy Example: Thread to P Data Mapping

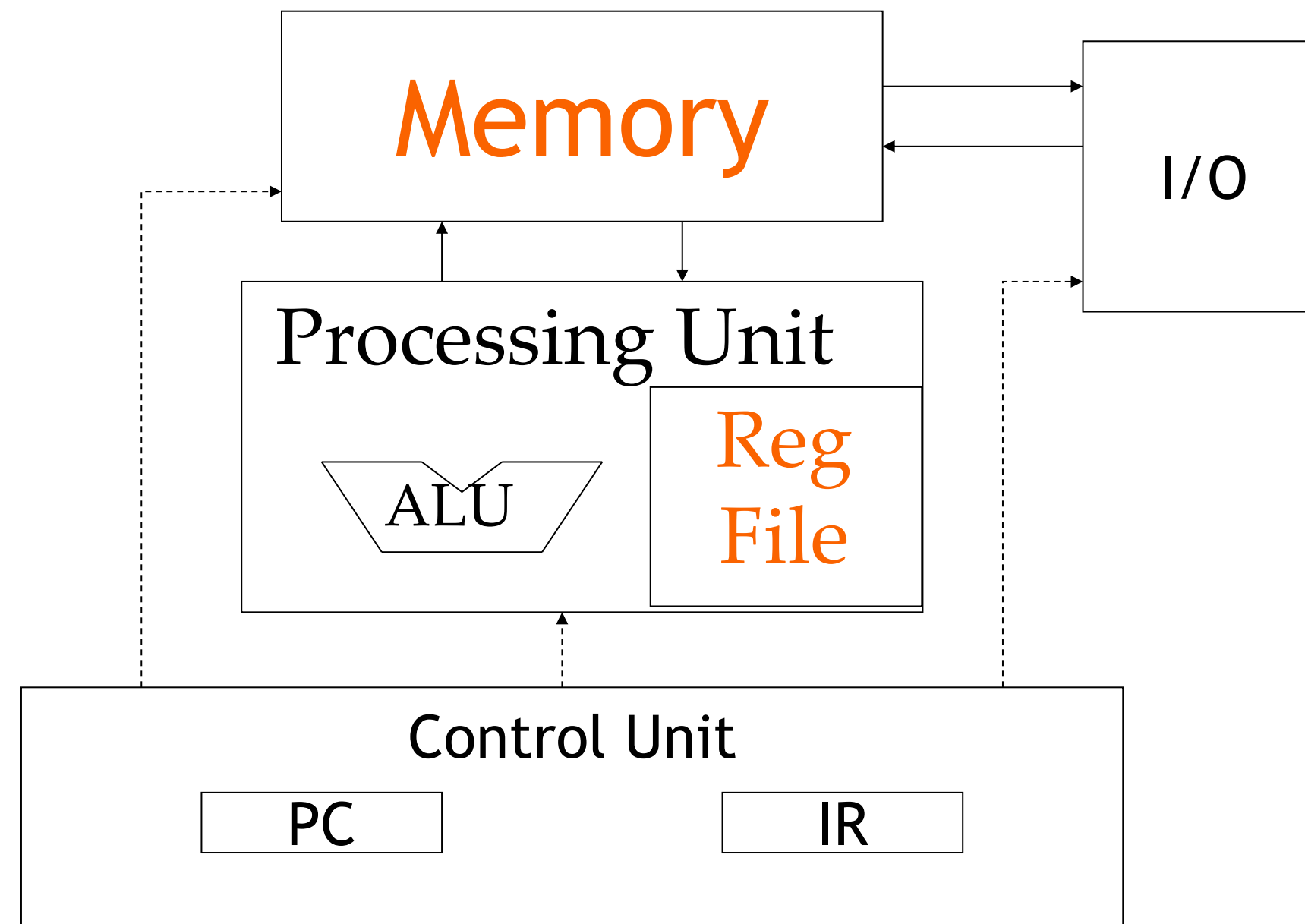


Calculation of $P_{0,0}$ and $P_{0,1}$

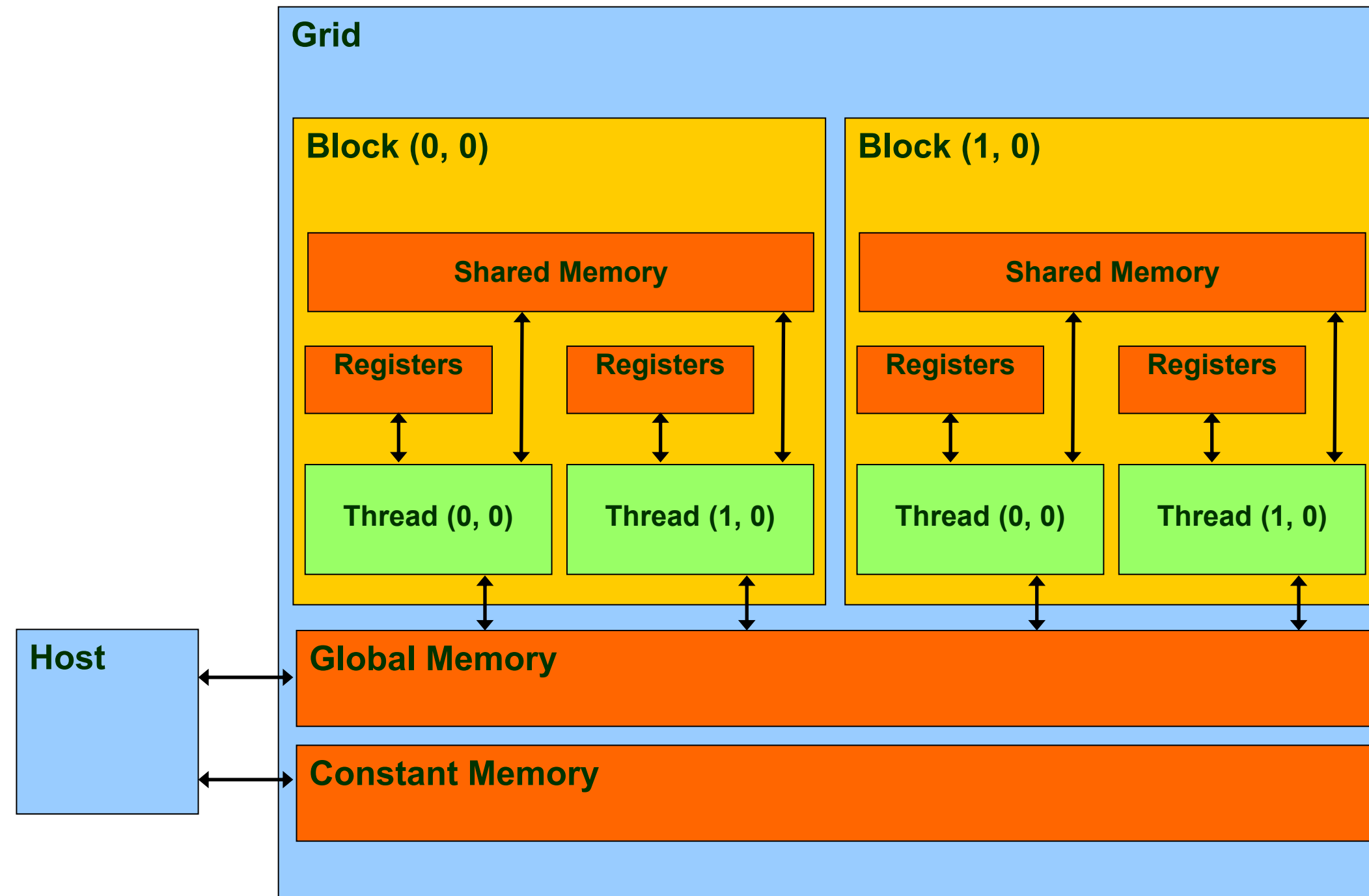
- Two 4B memory accesses per floating point multiply and add
- Arithmetic intensity is again $\frac{1}{4}$ FLOPS/B
- This is heavily memory bound.



Memory and Registers in the Von-Neumann Model



Programmer View of CUDA Memories



Declaring CUDA Variables

Variable declaration	Memory	Scope	Lifetime
<code>int LocalVar;</code>	register	thread	thread
<code>__device__ __shared__ int SharedVar;</code>	shared	block	block
<code>__device__ int GlobalVar;</code>	global	grid	application
<code>__device__ __constant__ int ConstantVar;</code>	constant	grid	application

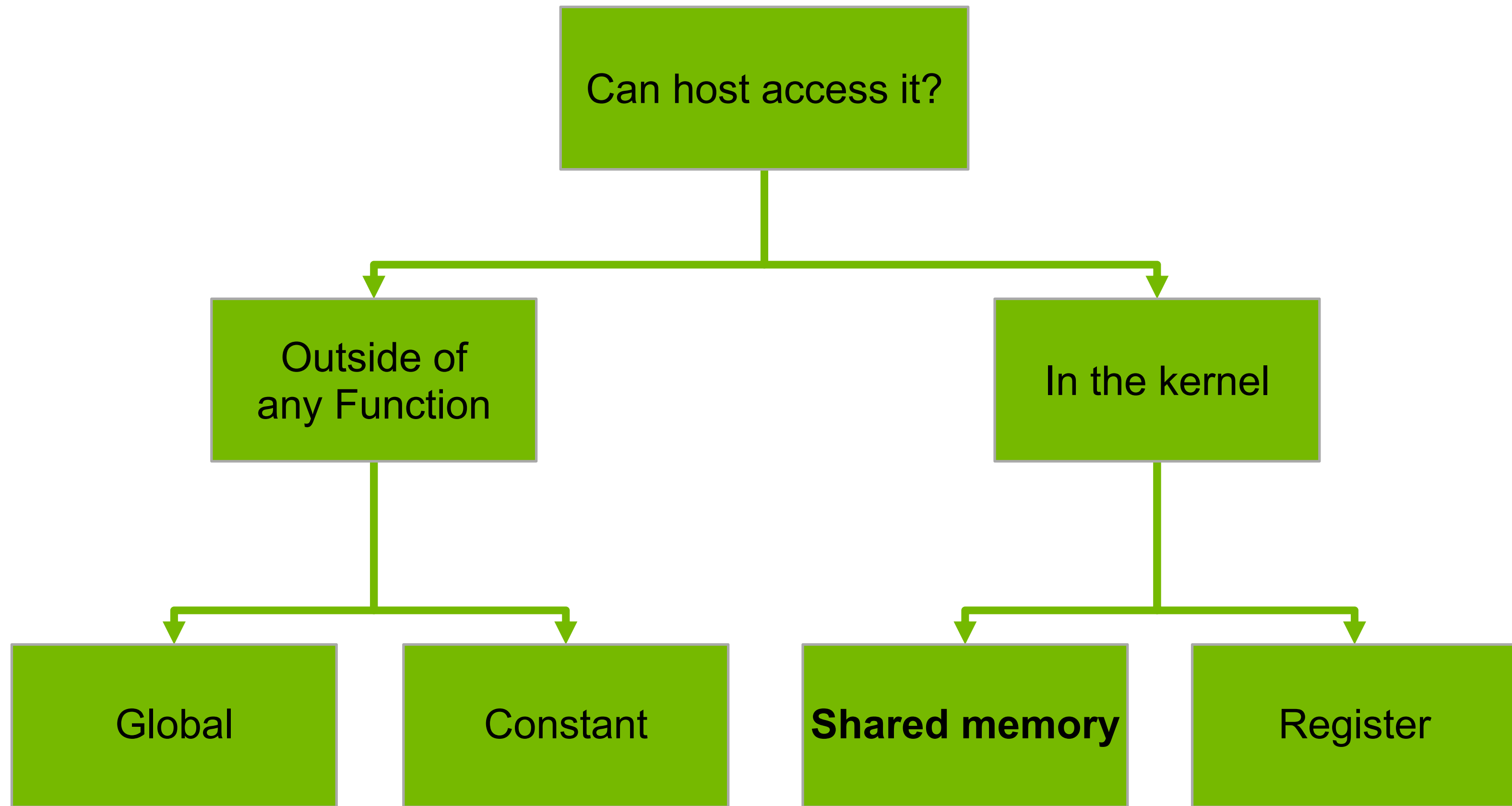
- `__device__` is optional when used with `__shared__`, or `__constant__`
- Automatic variables reside in a register
 - Except (big) per-thread arrays that spill over to off-chip (local) memory
- Only registers can be accessed without load/store instructions, thus bearing less overhead, everything else requires said additional instructions

Example:

Shared Memory Variable Declaration

```
void blurKernel(unsigned char * in, unsigned char * out, int w, int h)
{
    __shared__ float ds_in[TILE_WIDTH][TILE_WIDTH];
    ...
}
```

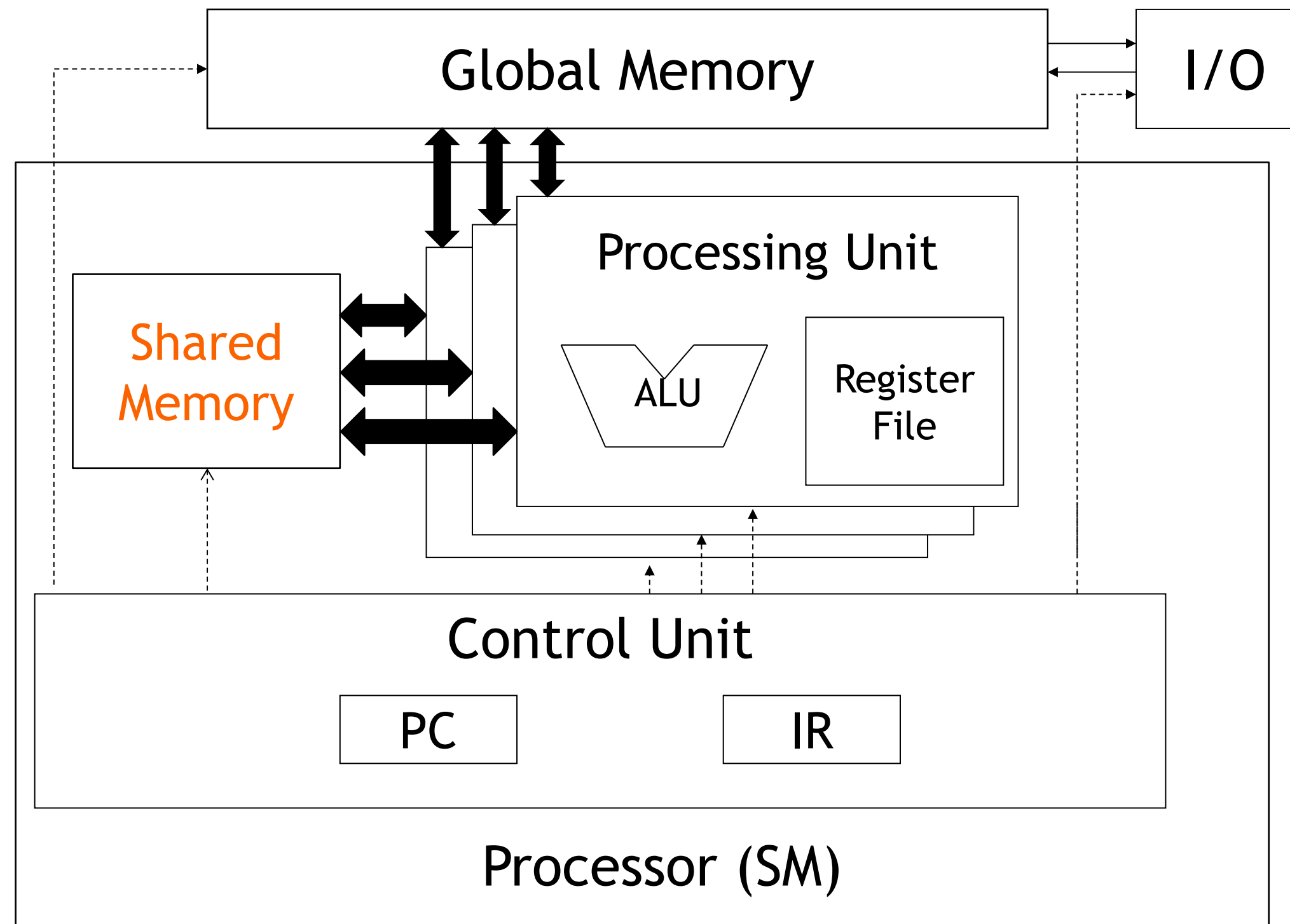
Where to Declare Variables?



Shared Memory in CUDA

- A special type of memory whose contents are explicitly defined and used in the kernel source code
 - One in each SM
 - Accessed at much higher speed (in both latency and throughput) than global memory
 - Scope of access and sharing - thread blocks
 - Lifetime – thread block, contents will disappear after all corresponding threads terminate execution
 - Accessed by memory load/store instructions (thereby having a bit of overhead compared to registers)
 - A form of scratchpad memory in computer architecture

Hardware View of CUDA Memories

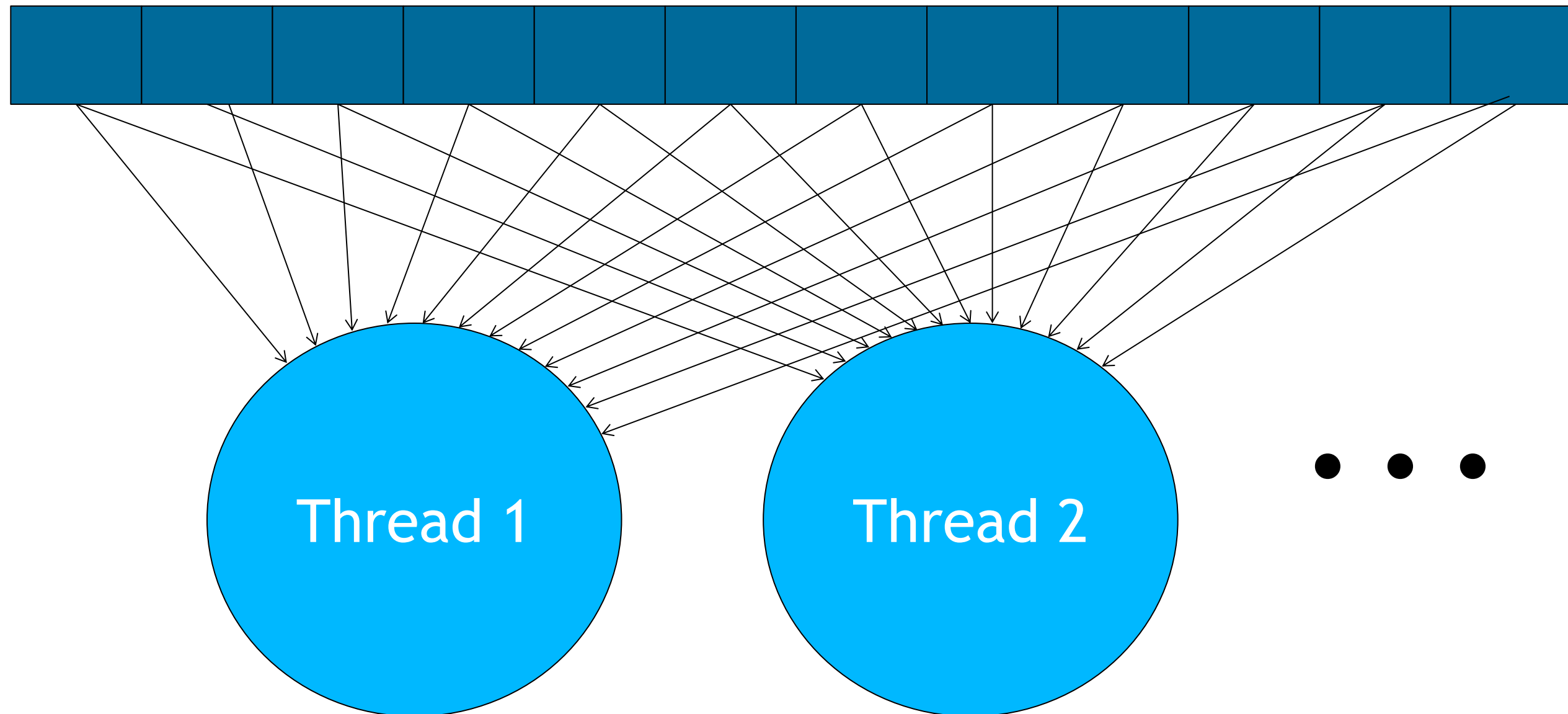


Objective

- To understand the motivation and ideas for tiled parallel algorithms
 - Reducing the limiting effect of memory bandwidth on parallel kernel performance
 - Tiled algorithms and barrier synchronization

Overlapping Global Memory Access Pattern

Global Memory

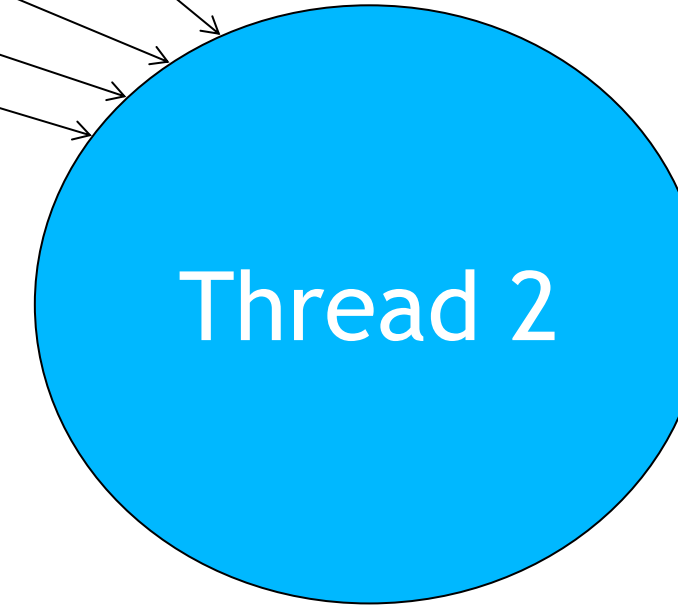
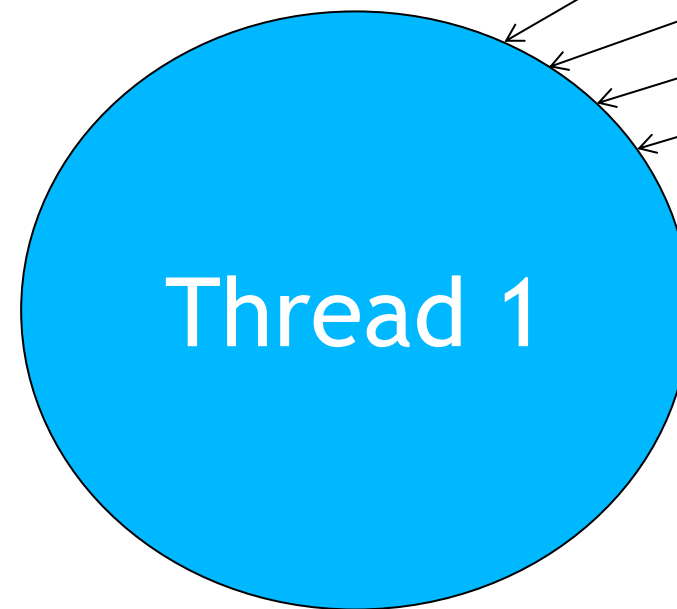
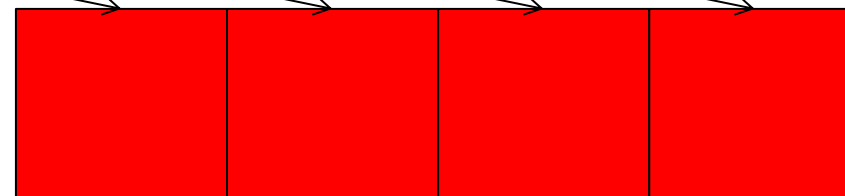


Tiling/Blocking - Basic Idea

Global Memory



On-chip Memory



...

Divide the global memory content into tiles

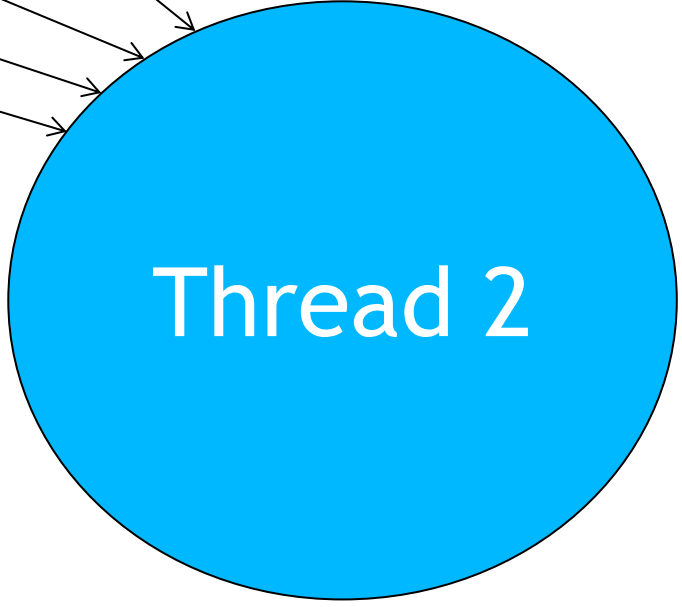
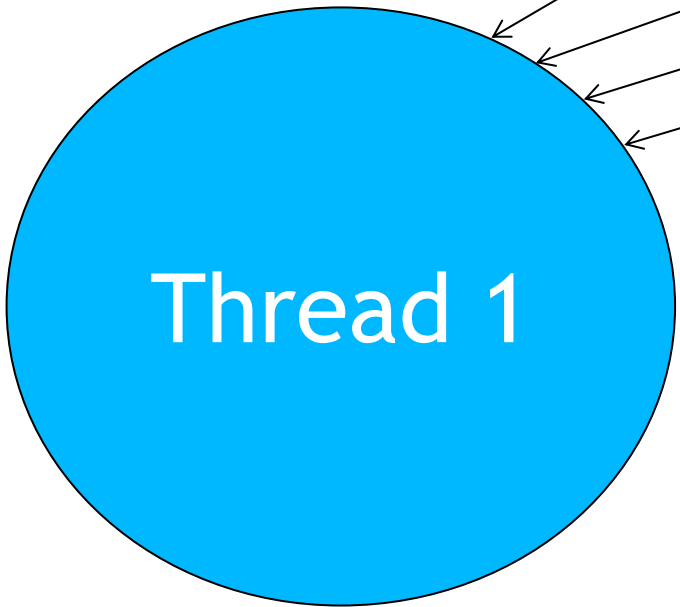
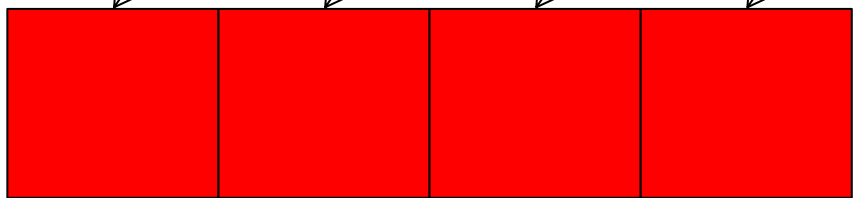
Focus the computation of threads on one or a small number of tiles at each point in time

Tiling/Blocking - Basic Idea

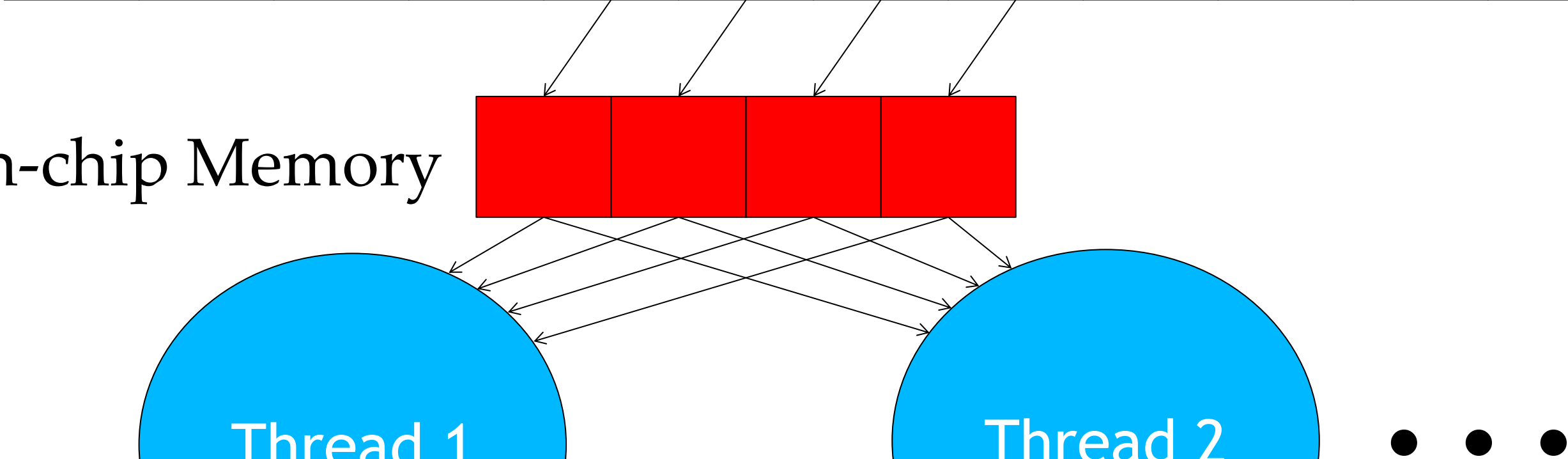
Global Memory



On-chip Memory

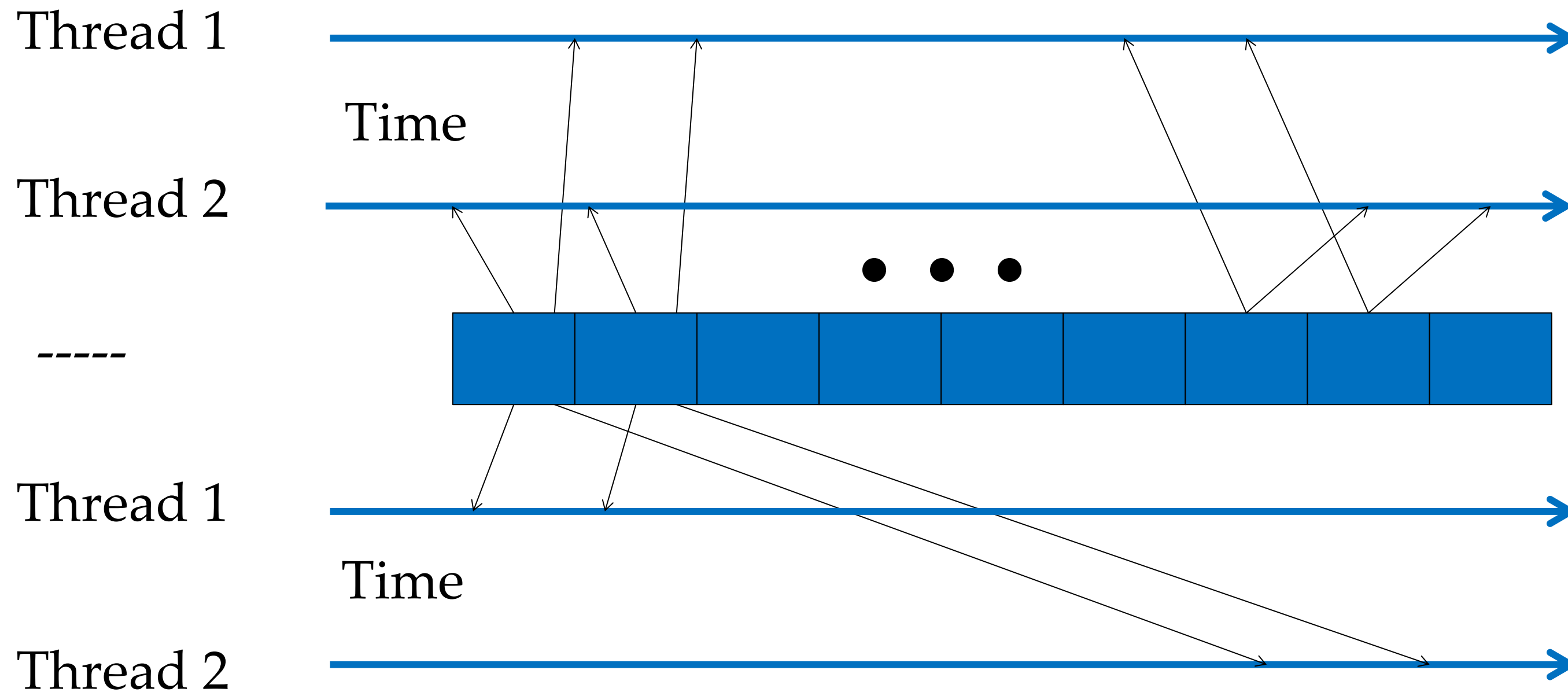


...



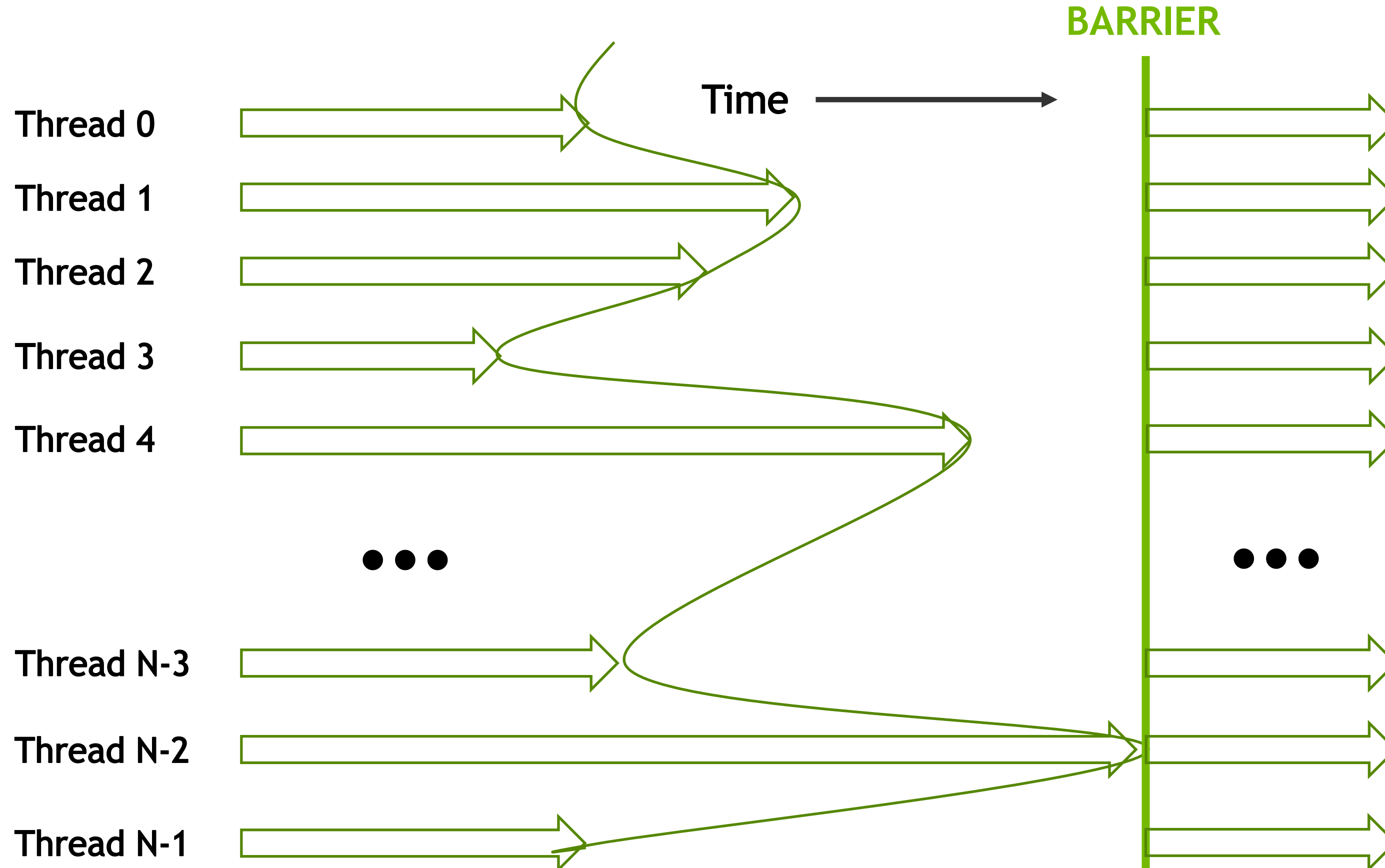
Basic Concept of Tiling

- Good: when threads have similar (overlapping) access patterns



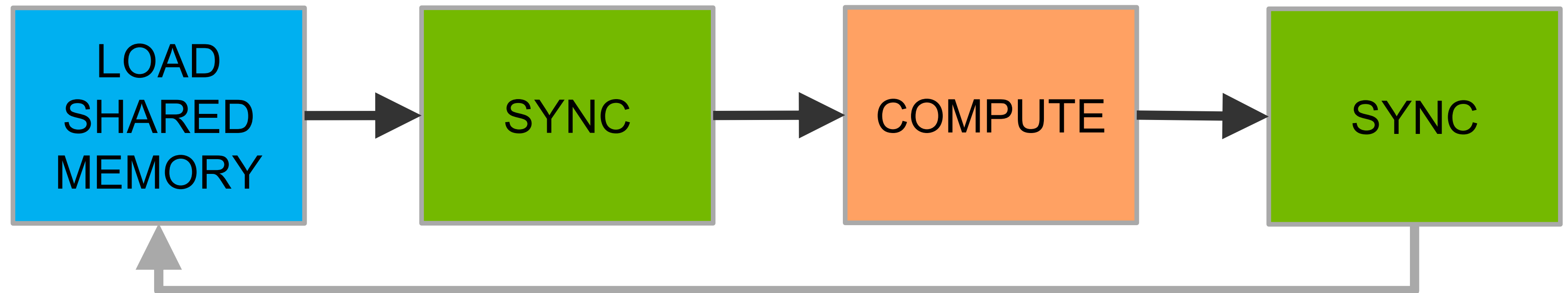
- Bad: when threads have very different access patterns

Barrier Synchronization for Tiling



Outline of Tiling Technique

- Identify a tile of global memory contents that are accessed by multiple threads
- Load the tile from global memory into on-chip memory
- Use barrier synchronization to make sure that all threads are ready to start the phase
- Have the multiple threads to access their data from the on-chip memory
- Use barrier synchronization to make sure that all threads have completed the current phase
- Move on to the next tile



REPEAT FOR THE NEXT TILE

Each repetition is called PHASE

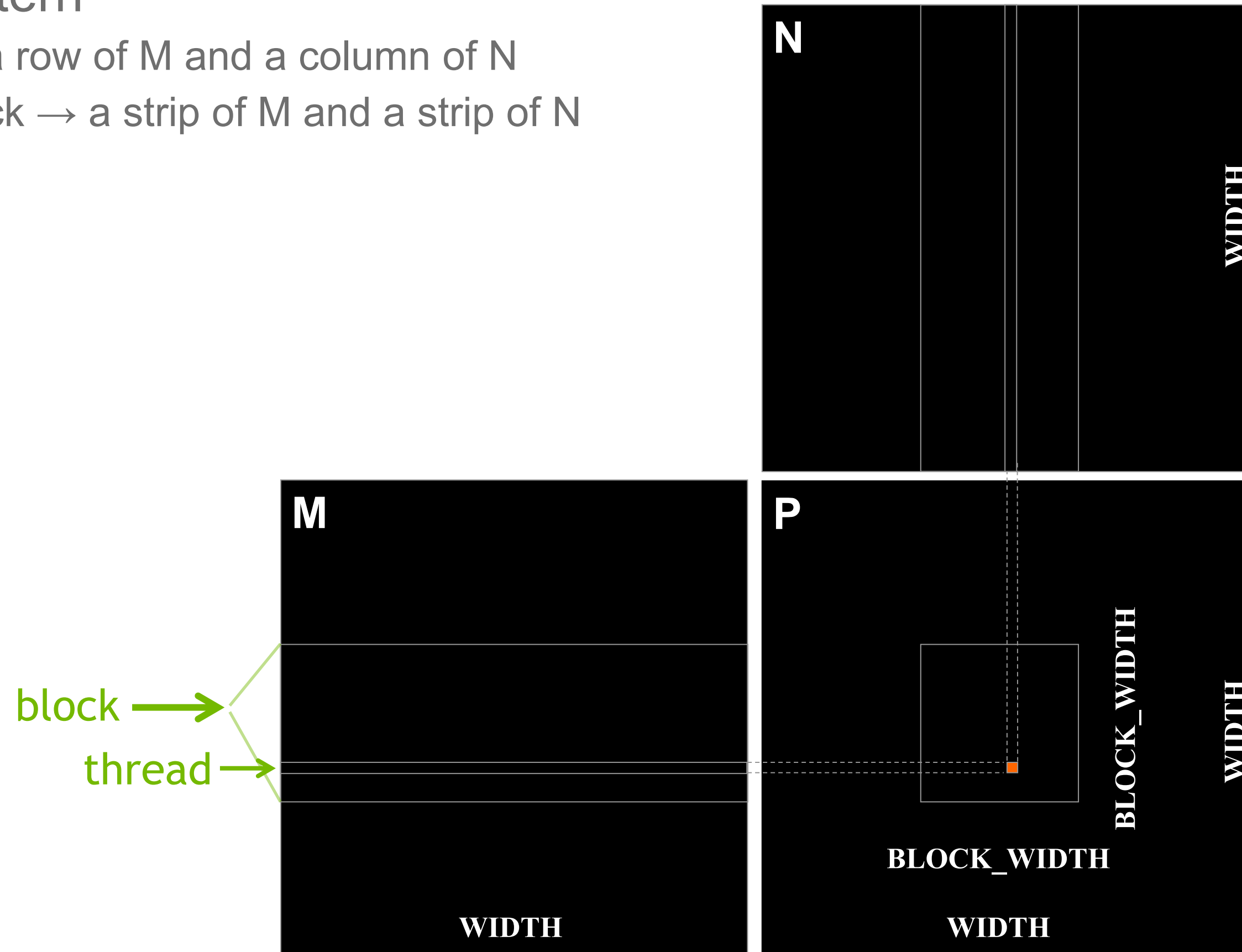
Objective

- To understand the design of a tiled parallel algorithm for matrix multiplication
 - Loading a tile
 - Phased execution
 - Barrier Synchronization

Matrix Multiplication

- Data access pattern
 - Each thread $\rightarrow$ a row of M and a column of N
 - Each thread block $\rightarrow$ a strip of M and a strip of N

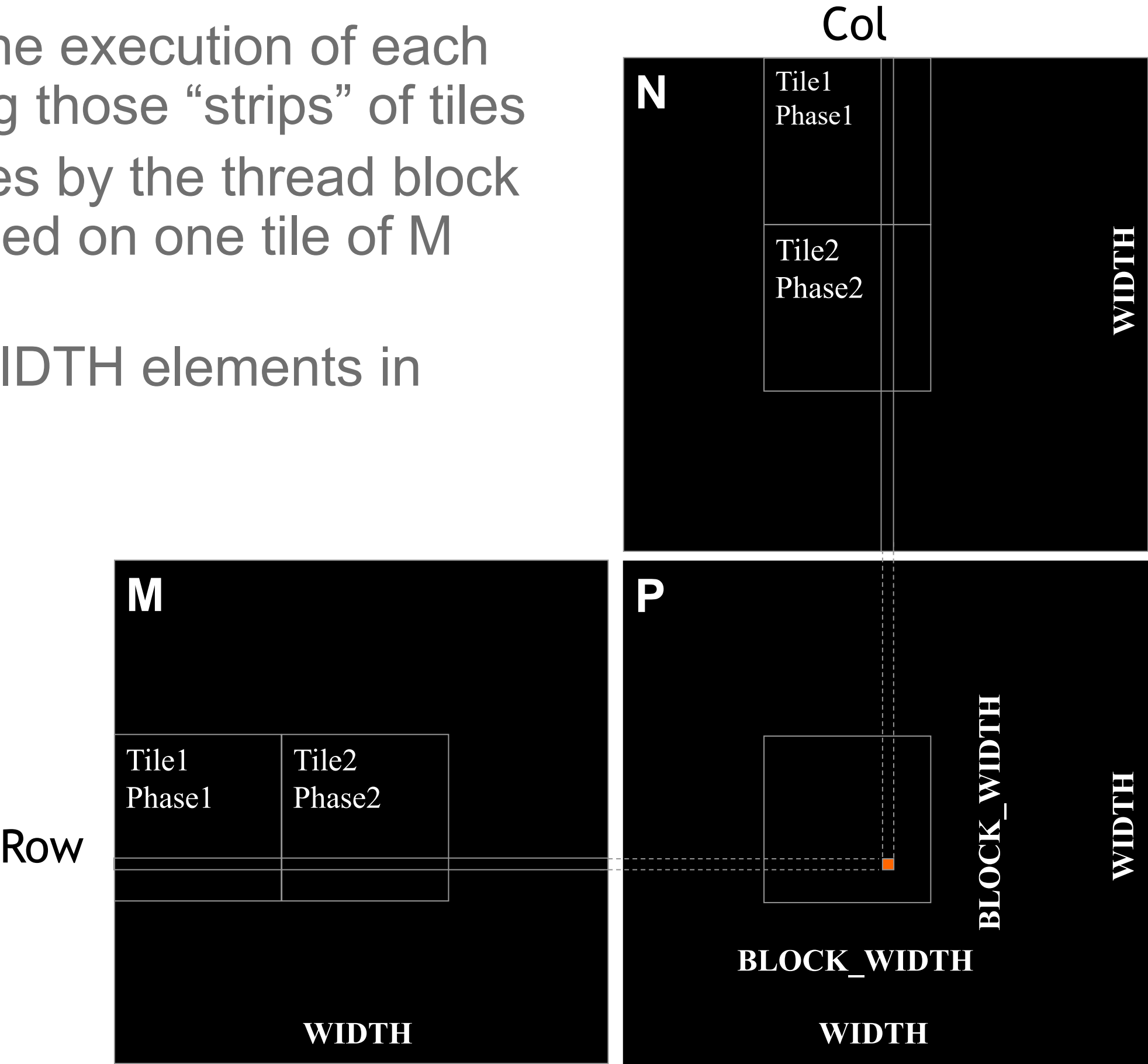
From now:
BLOCK_WIDTH
||
TILE_WIDTH



Tiled Matrix Multiplication

- Strip mining: break up the execution of each thread into phases along those “strips” of tiles
- so that the data accesses by the thread block in each phase are focused on one tile of M and one tile of N
- The tile is of BLOCK_WIDTH elements in each dimension

From now:
BLOCK_WIDTH
||
TILE_WIDTH



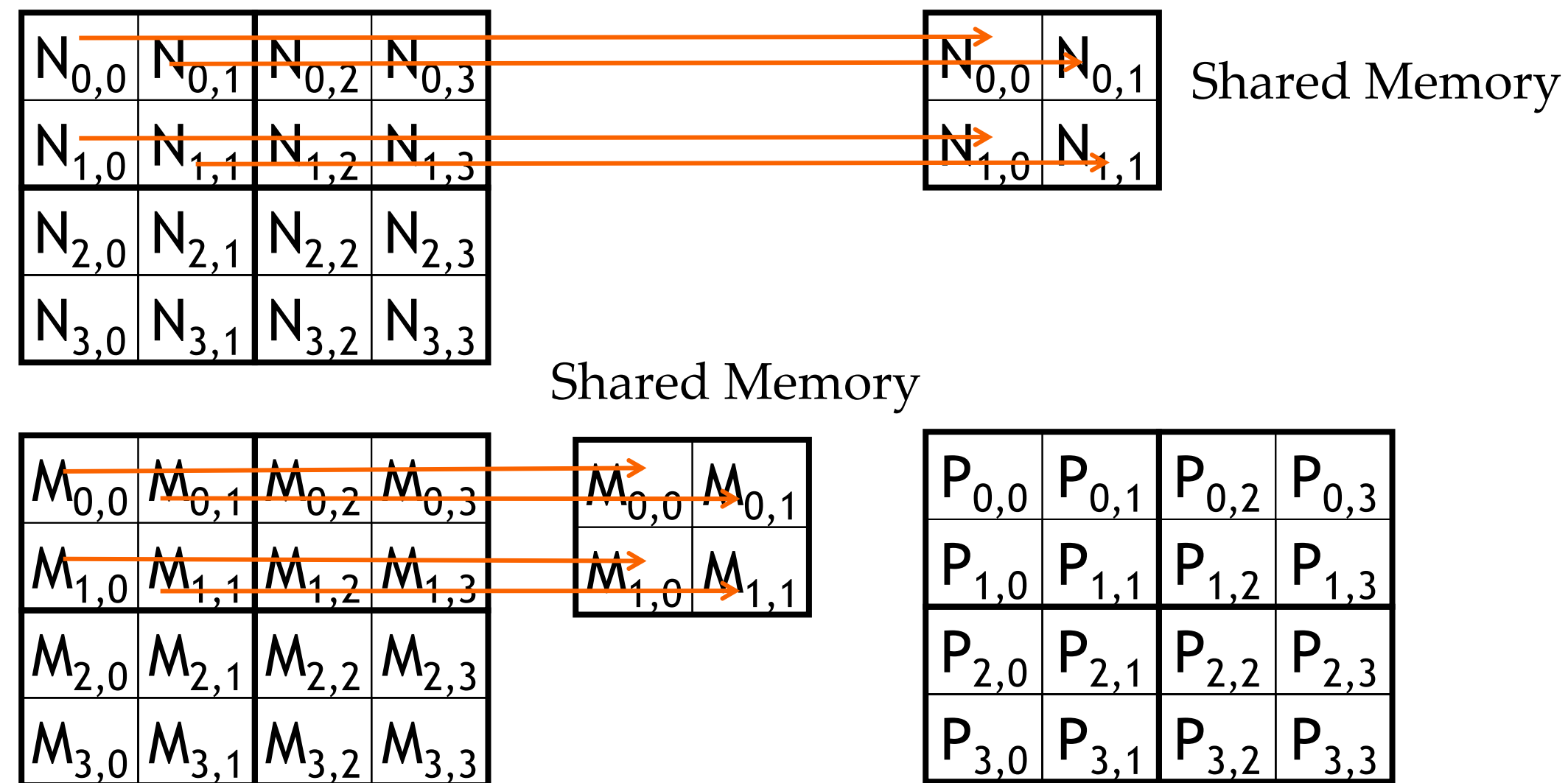
Loading a Tile

- All threads in a block participate
 - Each thread loads one M element and one N element in the block's shared memory
 - Synchronize
- Each block is in charge of an output tile, and that does not change while iterating along strips

Computing

- All threads compute a vector-vector product
 - Each thread continues its output element's partial sum along the newly available span of the inner dimension as of the loaded tiles
 - Synchronize

Phase 0 Load for Block (0,0)

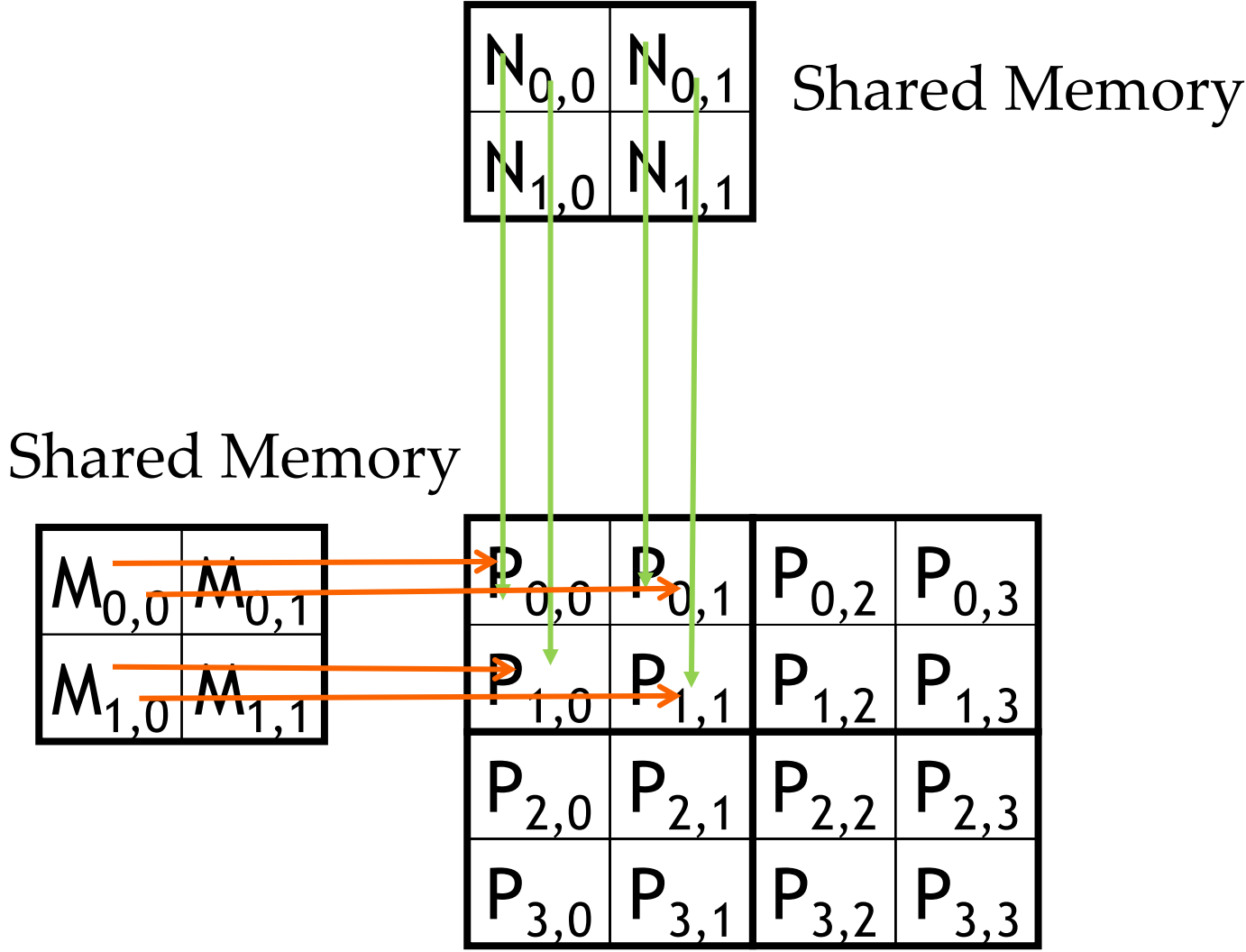


Recall: one thread per element of P.
Here we show only what happens in the first block
(top-left), all others do the same thing concurrently.

Phase 0 Use for Block (0,0) (iteration 0)

$N_{0,0}$	$N_{0,1}$	$N_{0,2}$	$N_{0,3}$
$N_{1,0}$	$N_{1,1}$	$N_{1,2}$	$N_{1,3}$
$N_{2,0}$	$N_{2,1}$	$N_{2,2}$	$N_{2,3}$
$N_{3,0}$	$N_{3,1}$	$N_{3,2}$	$N_{3,3}$

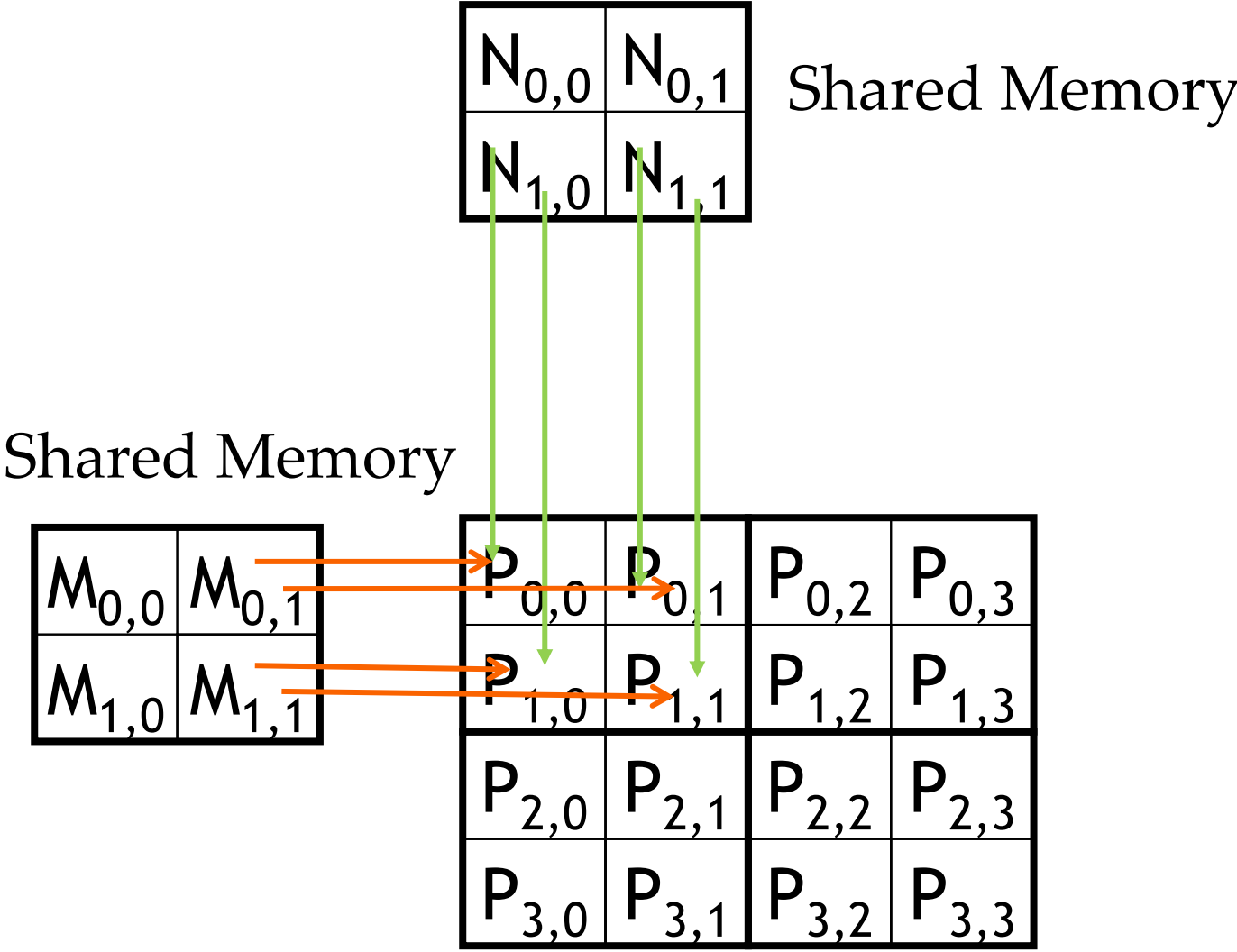
$M_{0,0}$	$M_{0,1}$	$M_{0,2}$	$M_{0,3}$
$M_{1,0}$	$M_{1,1}$	$M_{1,2}$	$M_{1,3}$
$M_{2,0}$	$M_{2,1}$	$M_{2,2}$	$M_{2,3}$
$M_{3,0}$	$M_{3,1}$	$M_{3,2}$	$M_{3,3}$



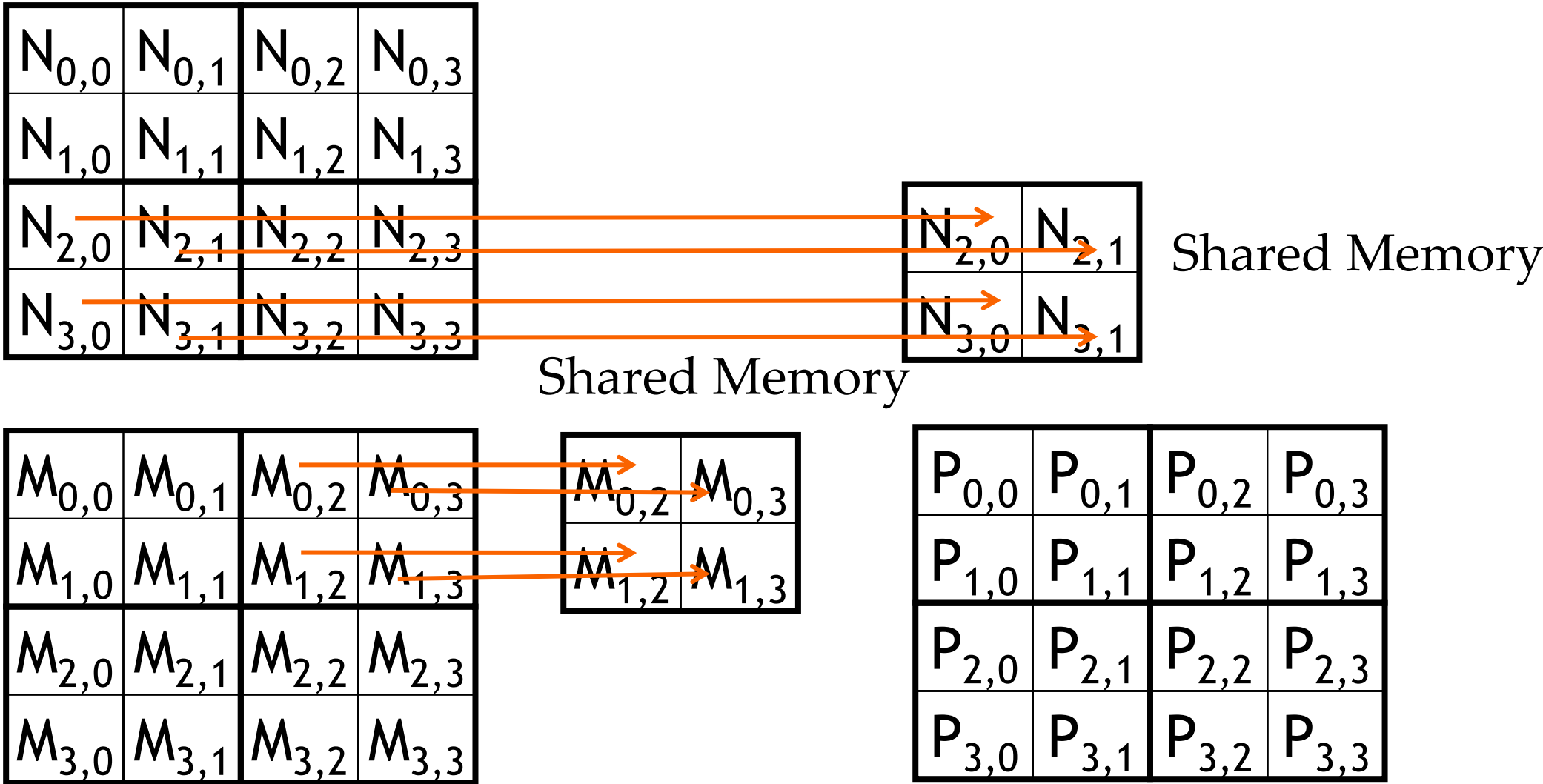
Phase 0 Use for Block (0,0) (iteration 1)

$N_{0,0}$	$N_{0,1}$	$N_{0,2}$	$N_{0,3}$
$N_{1,0}$	$N_{1,1}$	$N_{1,2}$	$N_{1,3}$
$N_{2,0}$	$N_{2,1}$	$N_{2,2}$	$N_{2,3}$
$N_{3,0}$	$N_{3,1}$	$N_{3,2}$	$N_{3,3}$

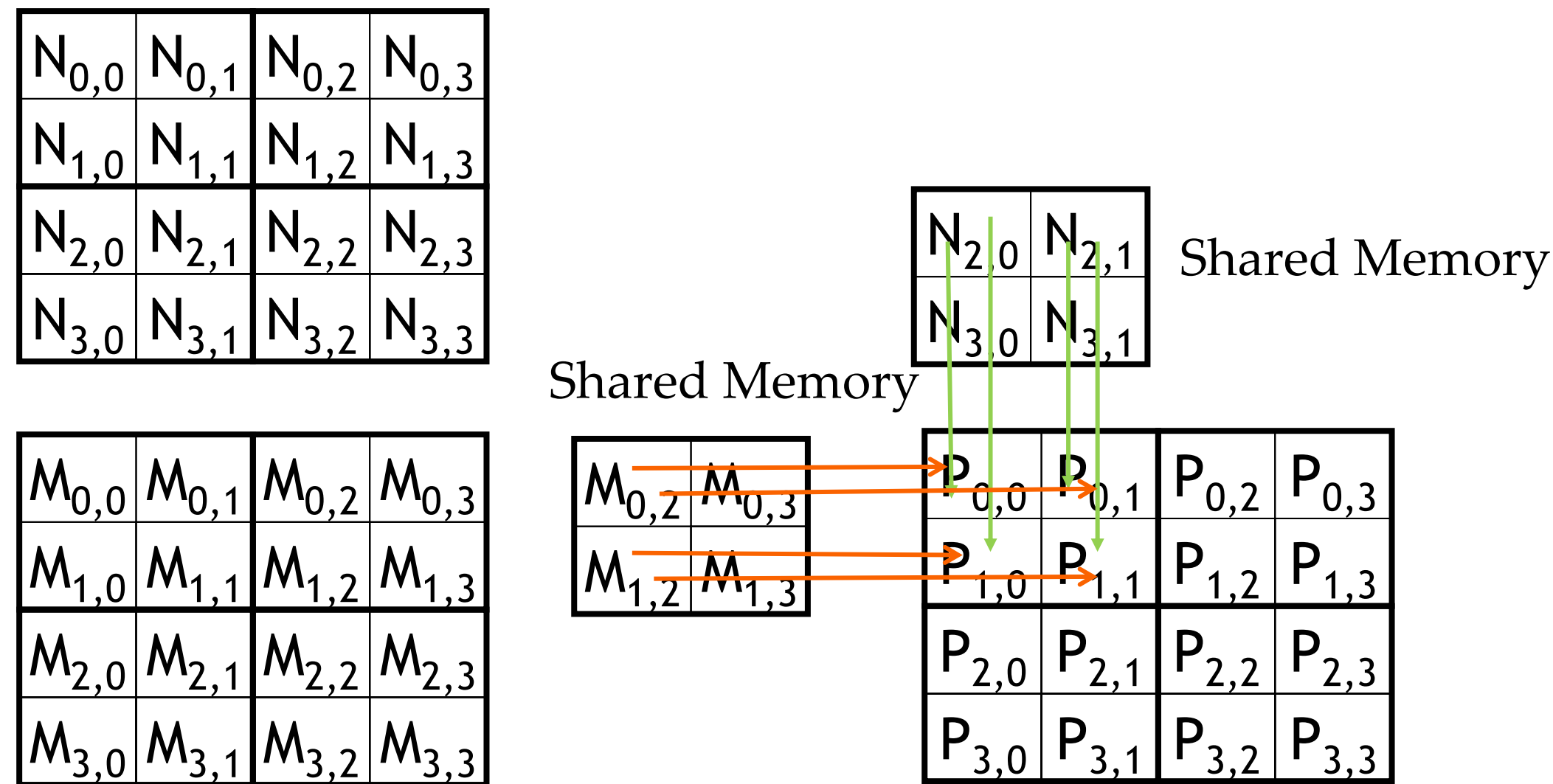
$M_{0,0}$	$M_{0,1}$	$M_{0,2}$	$M_{0,3}$
$M_{1,0}$	$M_{1,1}$	$M_{1,2}$	$M_{1,3}$
$M_{2,0}$	$M_{2,1}$	$M_{2,2}$	$M_{2,3}$
$M_{3,0}$	$M_{3,1}$	$M_{3,2}$	$M_{3,3}$



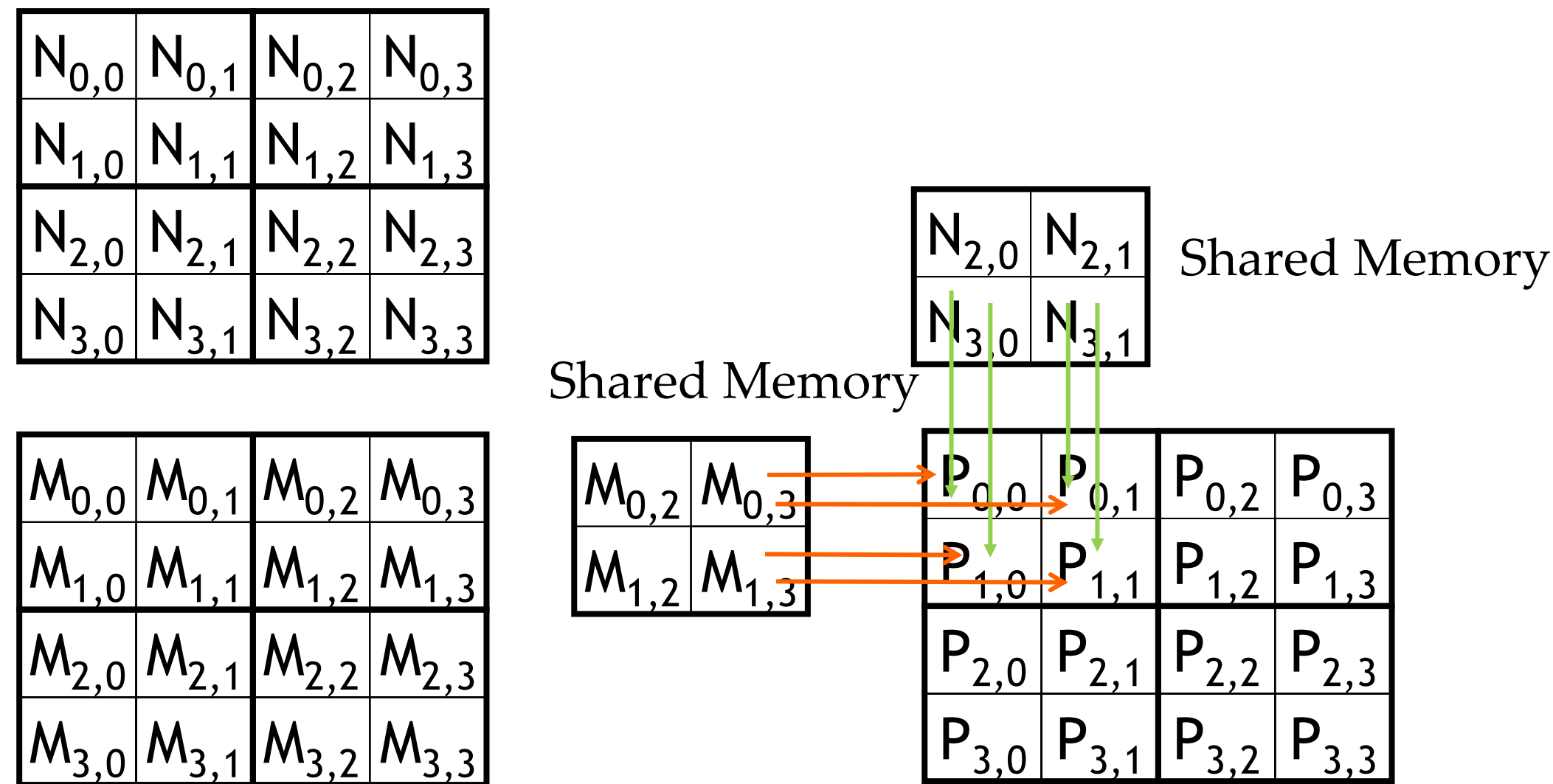
Phase 1 Load for Block (0,0)



Phase 1 Use for Block (0,0) (iteration 0)



Phase 1 Use for Block (0,0) (iteration 1)



Execution Phases of Toy Example

	Phase 0			Phase 1		
thread _{0,0}	M_{0,0} ↓ Mds _{0,0}	N_{0,0} ↓ Nds _{0,0}	PValue _{0,0} += Mds _{0,0} *Nds _{0,0} + Mds _{0,1} *Nds _{1,0}	M_{0,2} ↓ Mds _{0,0}	N_{2,0} ↓ Nds _{0,0}	PValue _{0,0} += Mds _{0,0} *Nds _{0,0} + Mds _{0,1} *Nds _{1,0}
thread _{0,1}	M_{0,1} ↓ Mds _{0,1}	N_{0,1} ↓ Nds _{1,0}	PValue _{0,1} += Mds _{0,0} *Nds _{0,1} + Mds _{0,1} *Nds _{1,1}	M_{0,3} ↓ Mds _{0,1}	N_{2,1} ↓ Nds _{0,1}	PValue _{0,1} += Mds _{0,0} *Nds _{0,1} + Mds _{0,1} *Nds _{1,1}
thread _{1,0}	M_{1,0} ↓ Mds _{1,0}	N_{1,0} ↓ Nds _{1,0}	PValue _{1,0} += Mds _{1,0} *Nds _{0,0} + Mds _{1,1} *Nds _{1,0}	M_{1,2} ↓ Mds _{1,0}	N_{3,0} ↓ Nds _{1,0}	PValue _{1,0} += Mds _{1,0} *Nds _{0,0} + Mds _{1,1} *Nds _{1,0}
thread _{1,1}	M_{1,1} ↓ Mds _{1,1}	N_{1,1} ↓ Nds _{1,1}	PValue _{1,1} += Mds _{1,0} *Nds _{0,1} + Mds _{1,1} *Nds _{1,1}	M_{1,3} ↓ Mds _{1,1}	N_{3,1} ↓ Nds _{1,1}	PValue _{1,1} += Mds _{1,0} *Nds _{0,1} + Mds _{1,1} *Nds _{1,1}

time 

Execution Phases of Toy Example (cont.)

	Phase 0			Phase 1		
thread _{0,0}	M_{0,0} ↓ Mds _{0,0}	N_{0,0} ↓ Nds _{0,0}	PValue _{0,0} += Mds _{0,0} *Nds _{0,0} + Mds _{0,1} *Nds _{1,0}	M_{0,2} ↓ Mds _{0,0}	N_{2,0} ↓ Nds _{0,0}	PValue _{0,0} += Mds _{0,0} *Nds _{0,0} + Mds _{0,1} *Nds _{1,0}
thread _{0,1}	M_{0,1} ↓ Mds _{0,1}	N_{0,1} ↓ Nds _{1,0}	PValue _{0,1} += Mds _{0,0} *Nds _{0,1} + Mds _{0,1} *Nds _{1,1}	M_{0,3} ↓ Mds _{0,1}	N_{2,1} ↓ Nds _{0,1}	PValue _{0,1} += Mds _{0,0} *Nds _{0,1} + Mds _{0,1} *Nds _{1,1}
thread _{1,0}	M_{1,0} ↓ Mds _{1,0}	N_{1,0} ↓ Nds _{1,0}	PValue _{1,0} += Mds _{1,0} *Nds _{0,0} + Mds _{1,1} *Nds _{1,0}	M_{1,2} ↓ Mds _{1,0}	N_{3,0} ↓ Nds _{1,0}	PValue _{1,0} += Mds _{1,0} *Nds _{0,0} + Mds _{1,1} *Nds _{1,0}
thread _{1,1}	M_{1,1} ↓ Mds _{1,1}	N_{1,1} ↓ Nds _{1,1}	PValue _{1,1} += Mds _{1,0} *Nds _{0,1} + Mds _{1,1} *Nds _{1,1}	M_{1,3} ↓ Mds _{1,1}	N_{3,1} ↓ Nds _{1,1}	PValue _{1,1} += Mds _{1,0} *Nds _{0,1} + Mds _{1,1} *Nds _{1,1}

time →

Shared memory allows each value to be accessed by multiple threads

Barrier Synchronization

- Synchronize all threads in a block
 - `__syncthreads()`
- All threads in the same block must reach the `__syncthreads()` before any of them can move on
- Best used to coordinate the phased execution tiled algorithms
 - To ensure that all elements of a tile are loaded at the beginning of a phase
 - To ensure that all elements of a tile are consumed at the end of a phase

Objective

- To learn to write a tiled matrix-multiplication kernel
 - Loading and using tiles for matrix multiplication
 - Barrier synchronization, shared memory
 - Resource Considerations
 - Assume that Width is a multiple of tile size for simplicity

Note: in these next slides we write `threadIdx.x` and `threadIdx.y` as `tx` and `ty` for brevity.

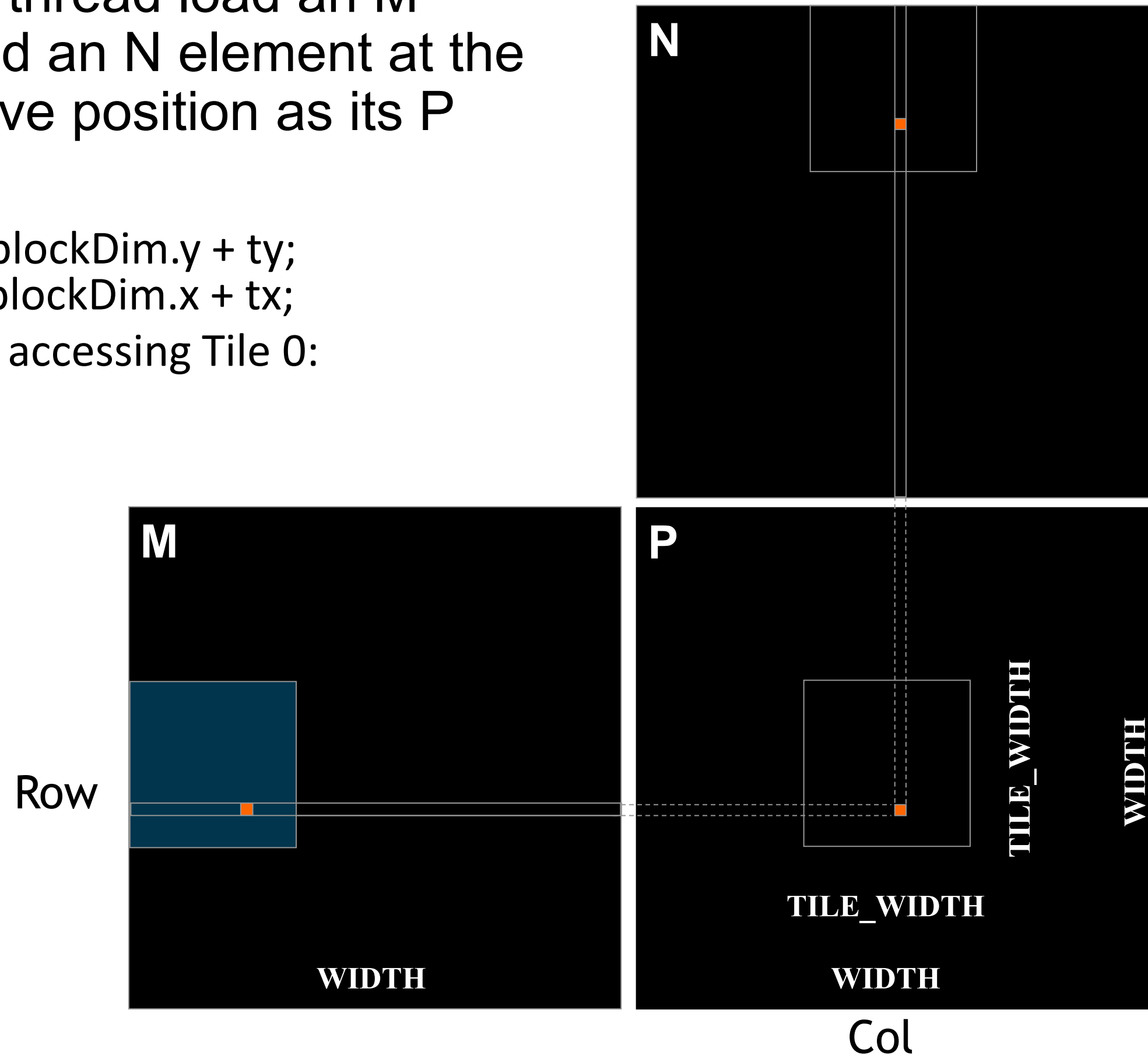
Loading Input Tile 0 of M (Phase 0)

- Have each thread load an M element and an N element at the same relative position as its P element.

```
int Row = by * blockDim.y + ty;  
int Col = bx * blockDim.x + tx;  
2D indexing for accessing Tile 0:
```

`M[Row][tx]`

`N[ty][Col]`

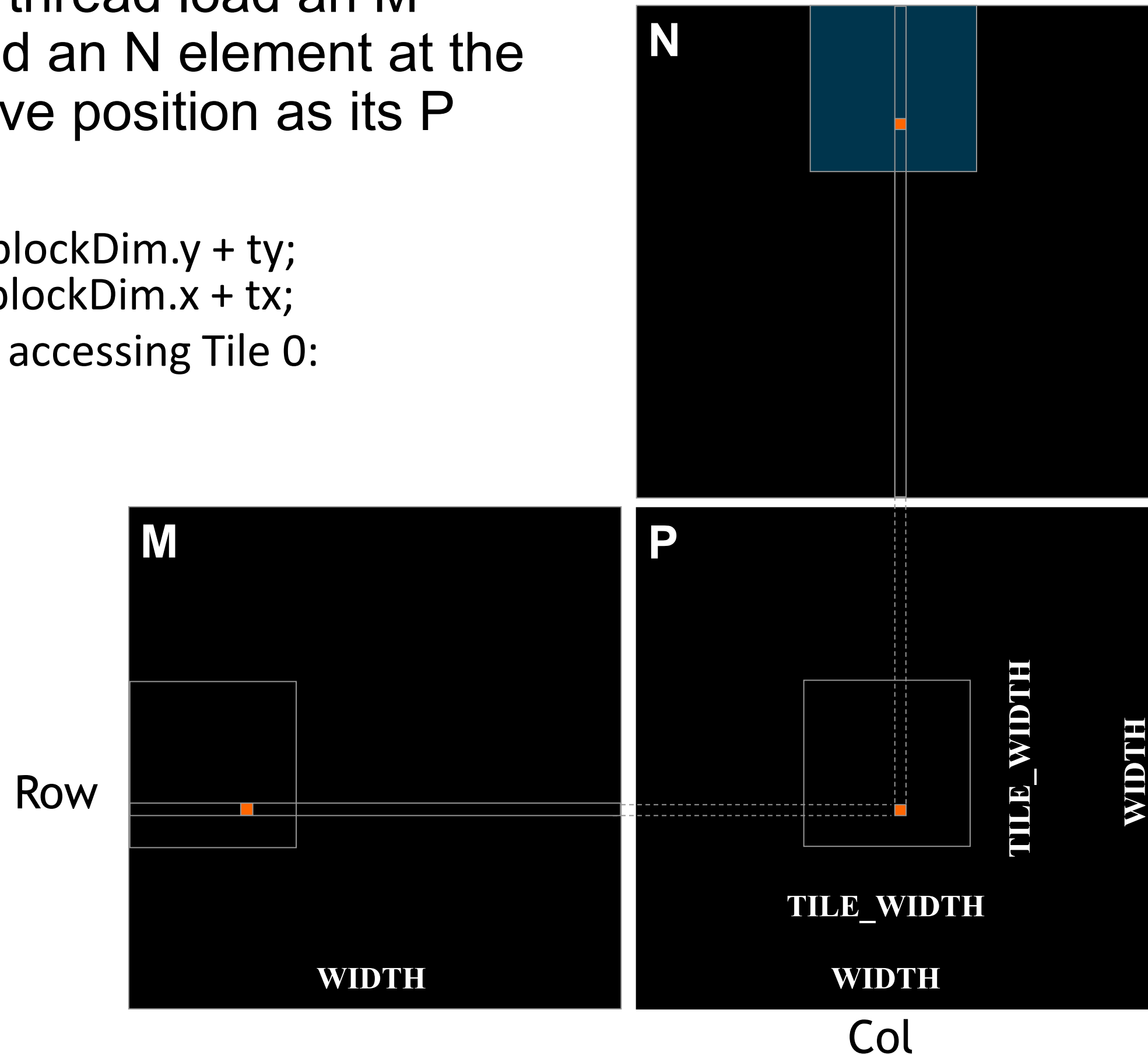


Loading Input Tile 0 of N (Phase 0)

- Have each thread load an M element and an N element at the same relative position as its P element.

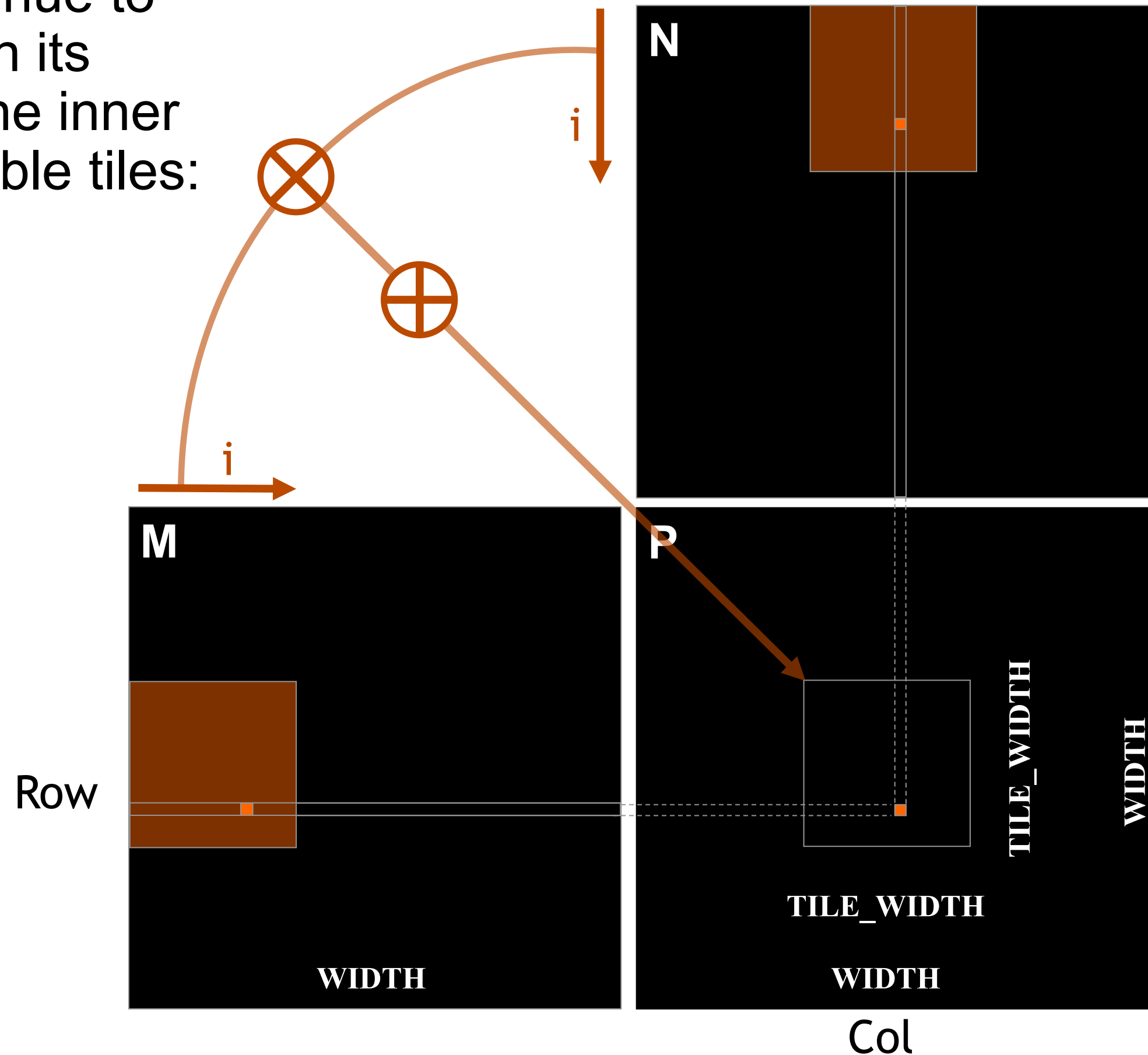
```
int Row = by * blockDim.y + ty;  
int Col = bx * blockDim.x + tx;  
2D indexing for accessing Tile 0:
```

```
M[Row][tx]  
N[ty][Col]
```



Compute on Tile 0 of P (Phase 0.1)

- Have each thread continue to accumulate products on its output by iterating on the inner dimension of the available tiles: $[0 \dots \text{TILE_WIDTH} - 1]$

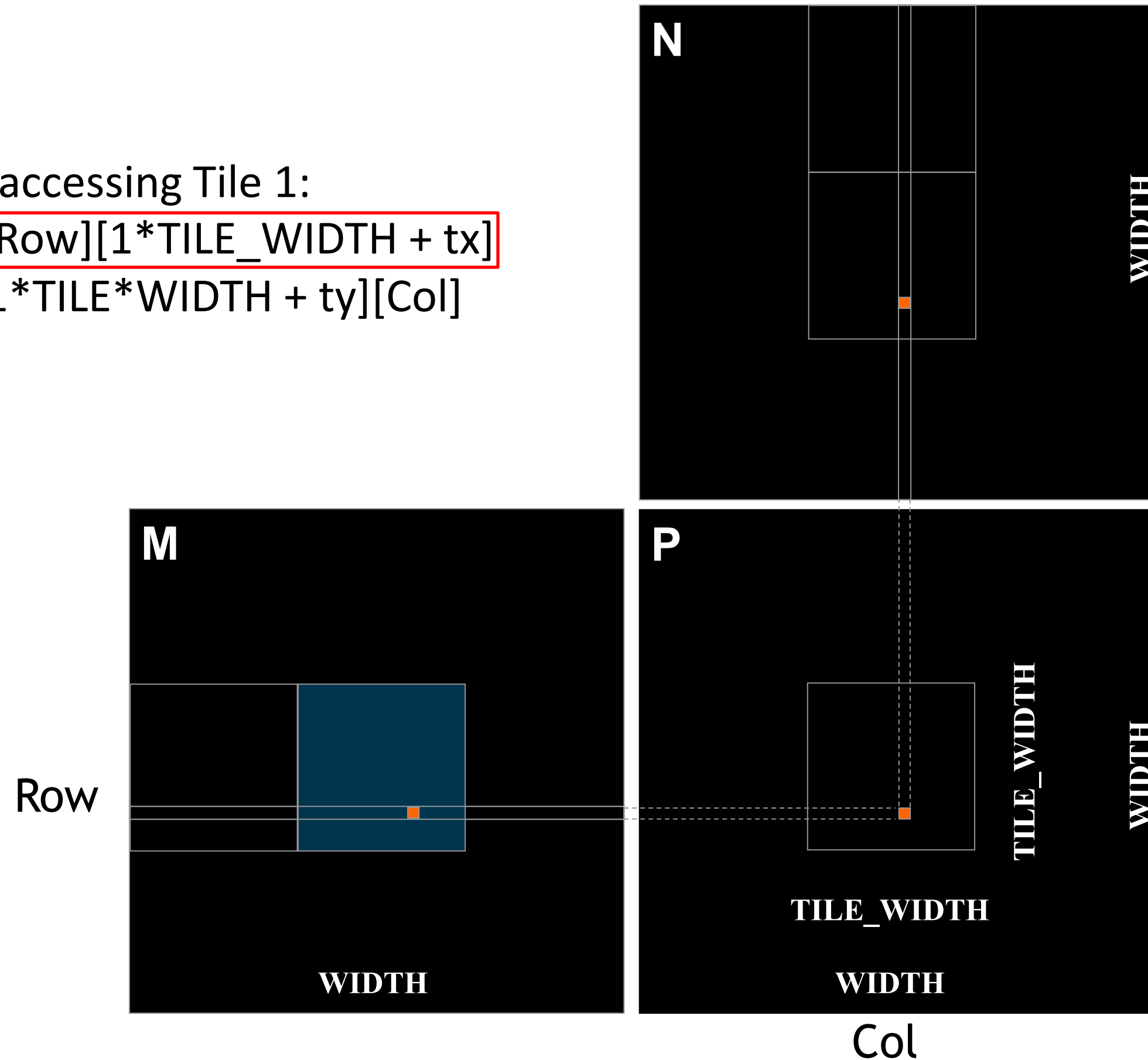


Loading Input Tile 1 of M (Phase 1)

2D indexing for accessing Tile 1:

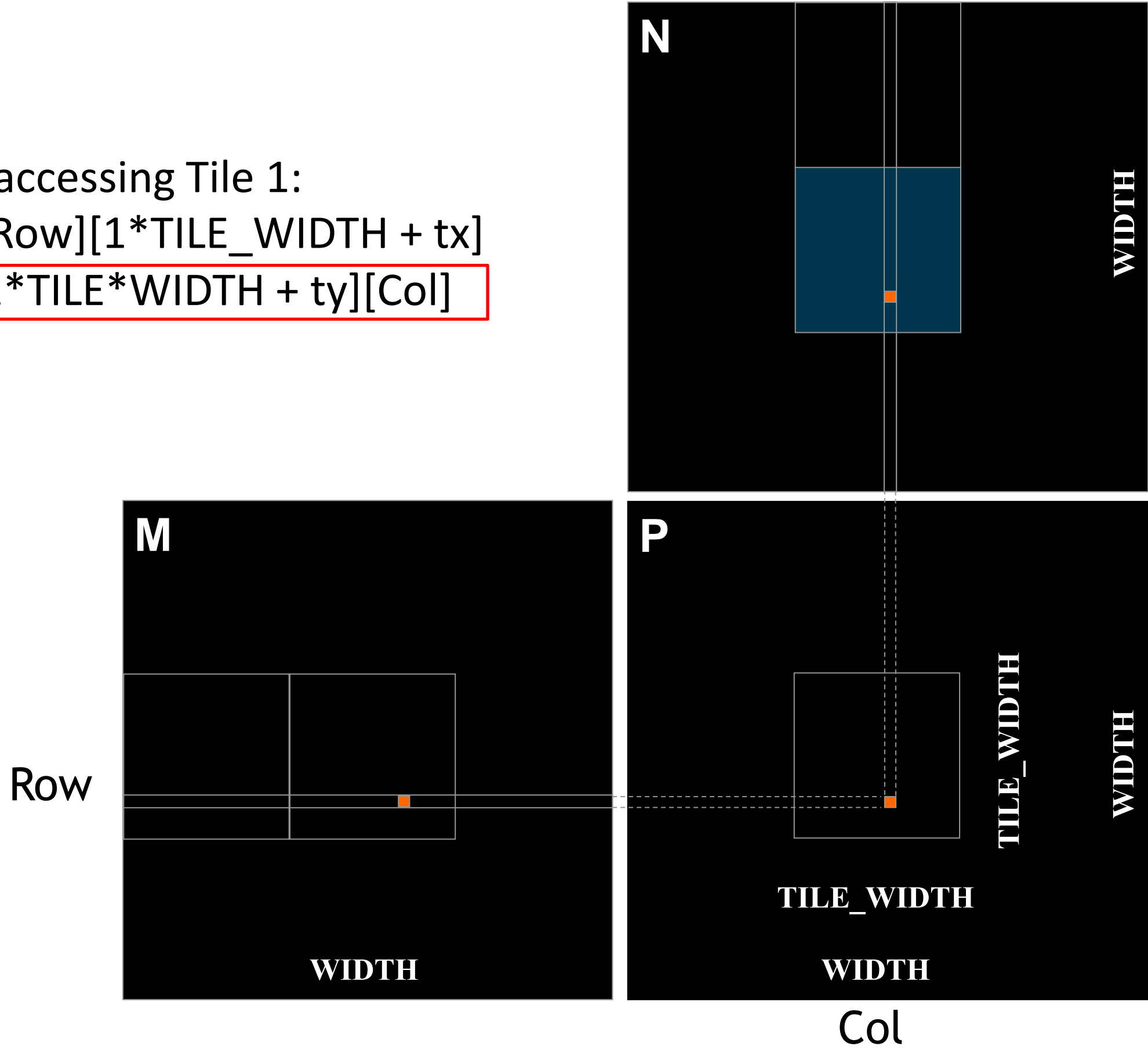
$M[\text{Row}][1 * \text{TILE_WIDTH} + \text{tx}]$

$N[1 * \text{TILE} * \text{WIDTH} + \text{ty}][\text{Col}]$

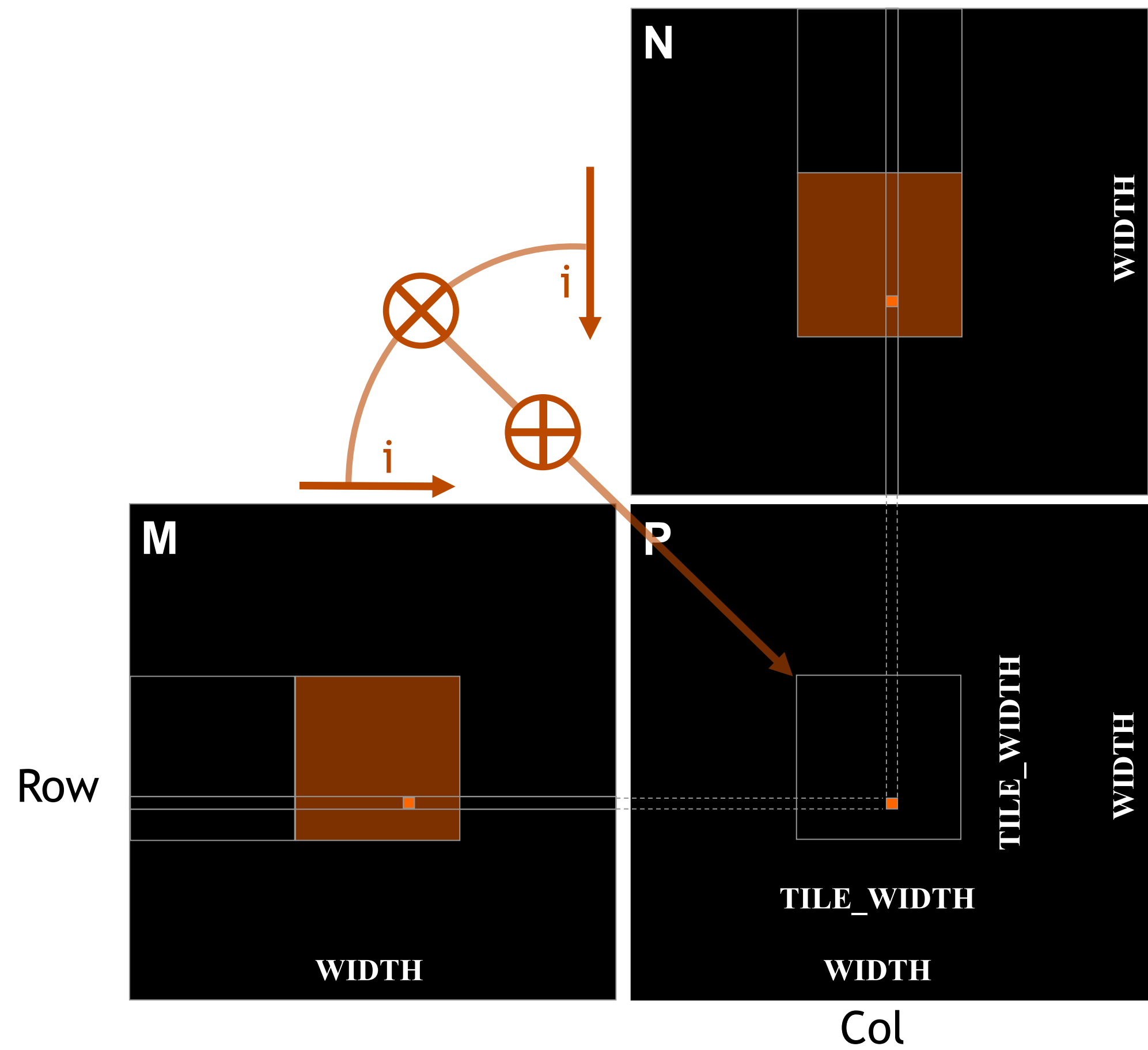


Loading Input Tile 1 of N (Phase 1)

2D indexing for accessing Tile 1:
M[Row][1*TILE_WIDTH + tx]
N[1*TILE*WIDTH + ty][Col]



Compute on Tile 1 of P (Phase 1.1)



M and N are dynamically allocated - use 1D indexing

This:

$M[\text{Row}][p * \text{TILE_WIDTH} + tx]$

Becomes this:

➡ $M[\text{Row} * \text{Width} + p * \text{TILE_WIDTH} + tx]$

This:

$N[p * \text{TILE_WIDTH} + ty][\text{Col}]$

Becomes this:

➡ $N[(p * \text{TILE_WIDTH} + ty) * \text{Width} + \text{Col}]$

Where 'p' is the sequence number of the current phase

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
    __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    int Row = by * blockDim.y + ty;
    int Col = bx * blockDim.x + tx;
    float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
    for (int p = 0; p < n/TILE_WIDTH; ++p) {
        // Collaborative loading of M and N tiles into shared memory
        ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
        ds_N[ty][tx] = N[(p*TILE_WIDTH+ty)*Width + Col];
        __syncthreads();

        for (int i = 0; i < TILE_WIDTH; ++i) Pvalue += ds_M[ty][i] * ds_N[i][tx];
        __syncthreads();
    }
    P[Row*Width+Col] = Pvalue;
}
```

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
    __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    int Row = by * blockDim.y + ty;
    int Col = bx * blockDim.x + tx;
    float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
    for (int p = 0; p < n/TILE_WIDTH; ++p) {
        // Collaborative loading of M and N tiles into shared memory
        ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
        ds_N[ty][tx] = N[(p*TILE_WIDTH+ty)*Width + Col];
        __syncthreads();

        for (int i = 0; i < TILE_WIDTH; ++i) Pvalue += ds_M[ty][i] * ds_N[i][tx];
        __syncthreads();
    }
    P[Row*Width+Col] = Pvalue;
}
```

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
    __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
    __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    int Row = by * blockDim.y + ty;
    int Col = bx * blockDim.x + tx;
    float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
    for (int p = 0; p < n/TILE_WIDTH; ++p) {
        // Collaborative loading of M and N tiles into shared memory
        ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
        ds_N[ty][tx] = N[(p*TILE_WIDTH+ty)*Width + Col];
        __syncthreads();

        for (int i = 0; i < TILE_WIDTH; ++i) Pvalue += ds_M[ty][i] * ds_N[i][tx];
        __syncthreads();
    }
    P[Row*Width+Col] = Pvalue;
}
```

Tile (Thread Block) Size Considerations

- Each **thread block** should have many threads
 - TILE_WIDTH of 16 gives $16 \times 16 = 256$ threads
 - TILE_WIDTH of 32 gives $32 \times 32 = 1024$ threads
- For TILE_WIDTH = 16, in each phase, each block performs $2 \times 256 = 512$ float loads from global memory for $256 \times (2 \times 16) = 8,192$ mul/add operations. (16 floating-point operations for each memory load)
- For TILE_WIDTH = 32, in each phase, each block performs $2 \times 1024 = 2048$ float loads from global memory for $1024 \times (2 \times 32) = 65,536$ mul/add operations. (32 floating-point operation for each memory load)

We successfully increased arithmetic intensity by $\text{TILE_WIDTH} \times$.

We thus need $1/\text{TILE_WIDTH}$ the global memory bandwidth for the same throughput!

Shared Memory and Threading

- For an SM with 16KB shared memory
 - Shared memory size is implementation dependent!
 - For `TILE_WIDTH = 16`, each thread block uses $2 \times 256 \times 4\text{B} = 2\text{KB}$ of shared memory.
 - For 16KB shared memory, one can potentially have up to 8 thread blocks executing
 - This allows up to $8 \times 512 = 4,096$ pending loads. (2 per thread, 256 threads per block)
 - The next `TILE_WIDTH 32` would lead to $2 \times 32 \times 32 \times 4 \text{ Byte} = 8\text{K Byte}$ shared memory usage per thread block, allowing 2 thread blocks active at the same time
 - However, in a GPU where the thread count is limited to 1536 threads per SM, the number of blocks per SM is reduced to one!
- Each `__syncthread()` can reduce the number of active threads for a block
 - More thread blocks can be advantageous
 - While a warp is waiting for something (usually memory) others may go ahead

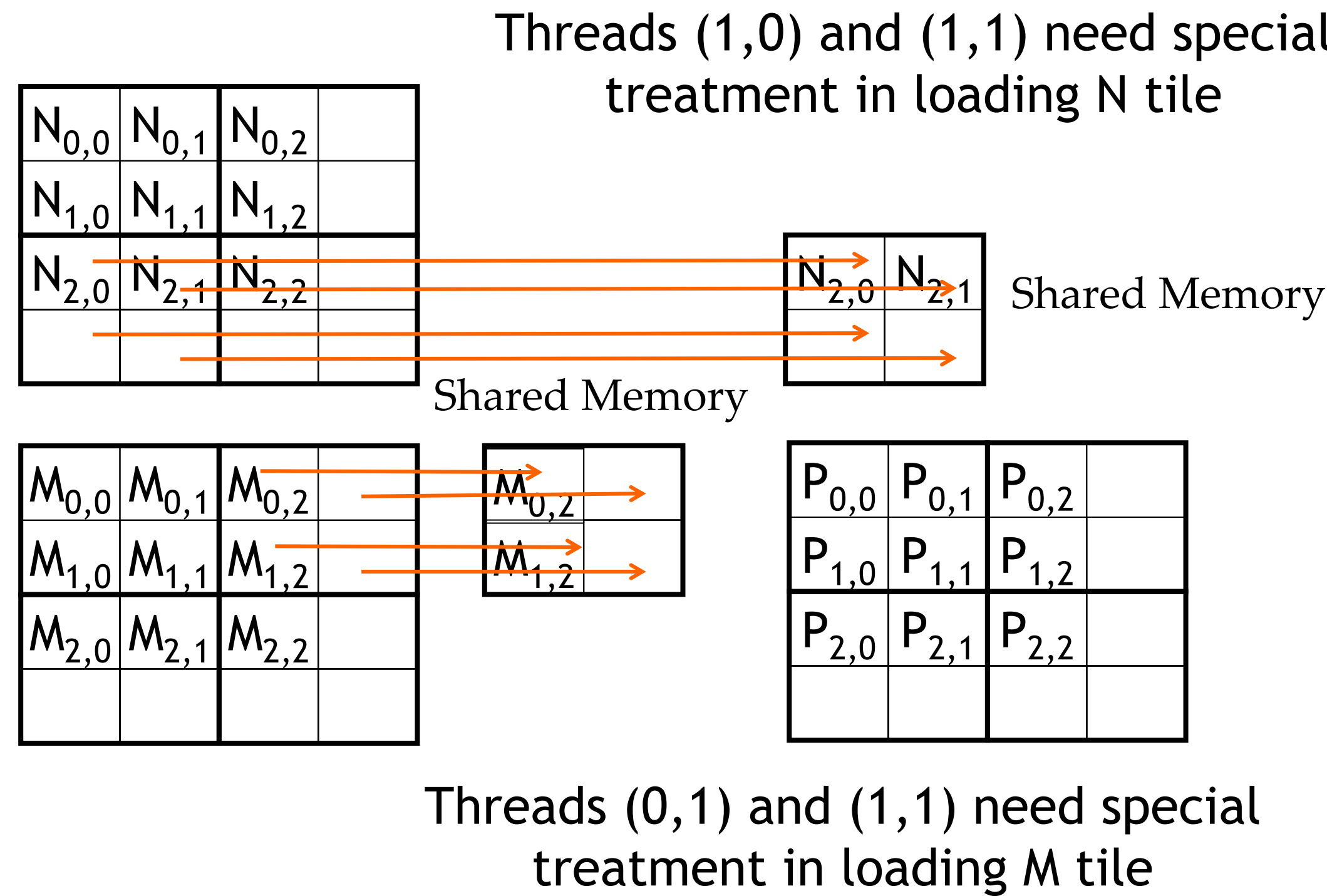
Objective

- To learn to handle arbitrary matrix sizes in tiled matrix multiplication
 - Boundary condition checking
 - Regularizing tile contents
 - Rectangular matrices

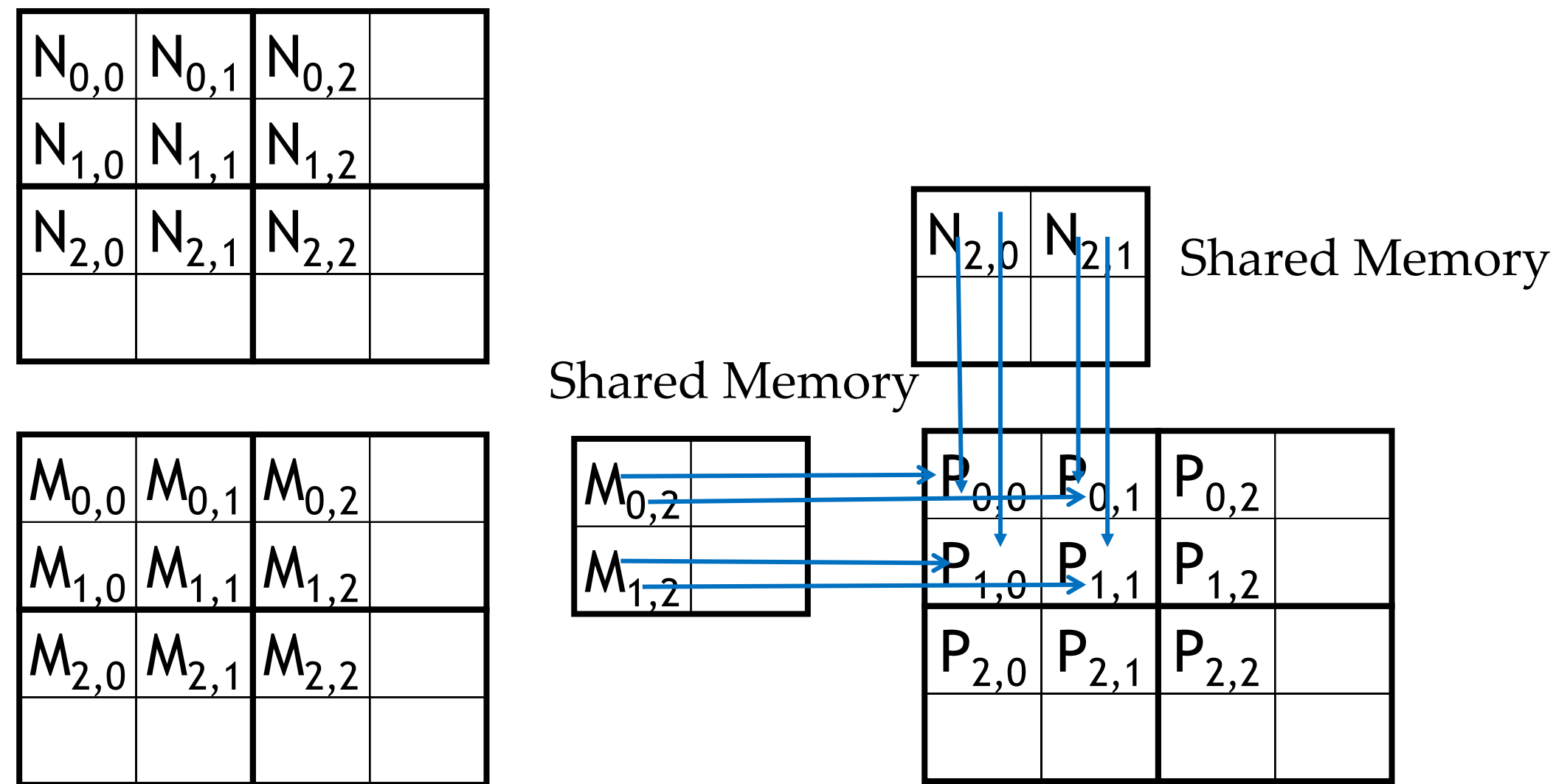
Handling Matrix of Arbitrary Size

- The tiled matrix multiplication kernel we presented so far can handle only square matrices whose dimensions (Width) are multiples of the tile width (TILE_WIDTH)
 - However, real applications need to handle arbitrary sized matrices.
 - One could pad (add elements to) the rows and columns into multiples of the tile size, but would have significant space and data transfer time overhead.
- We will take a different approach.

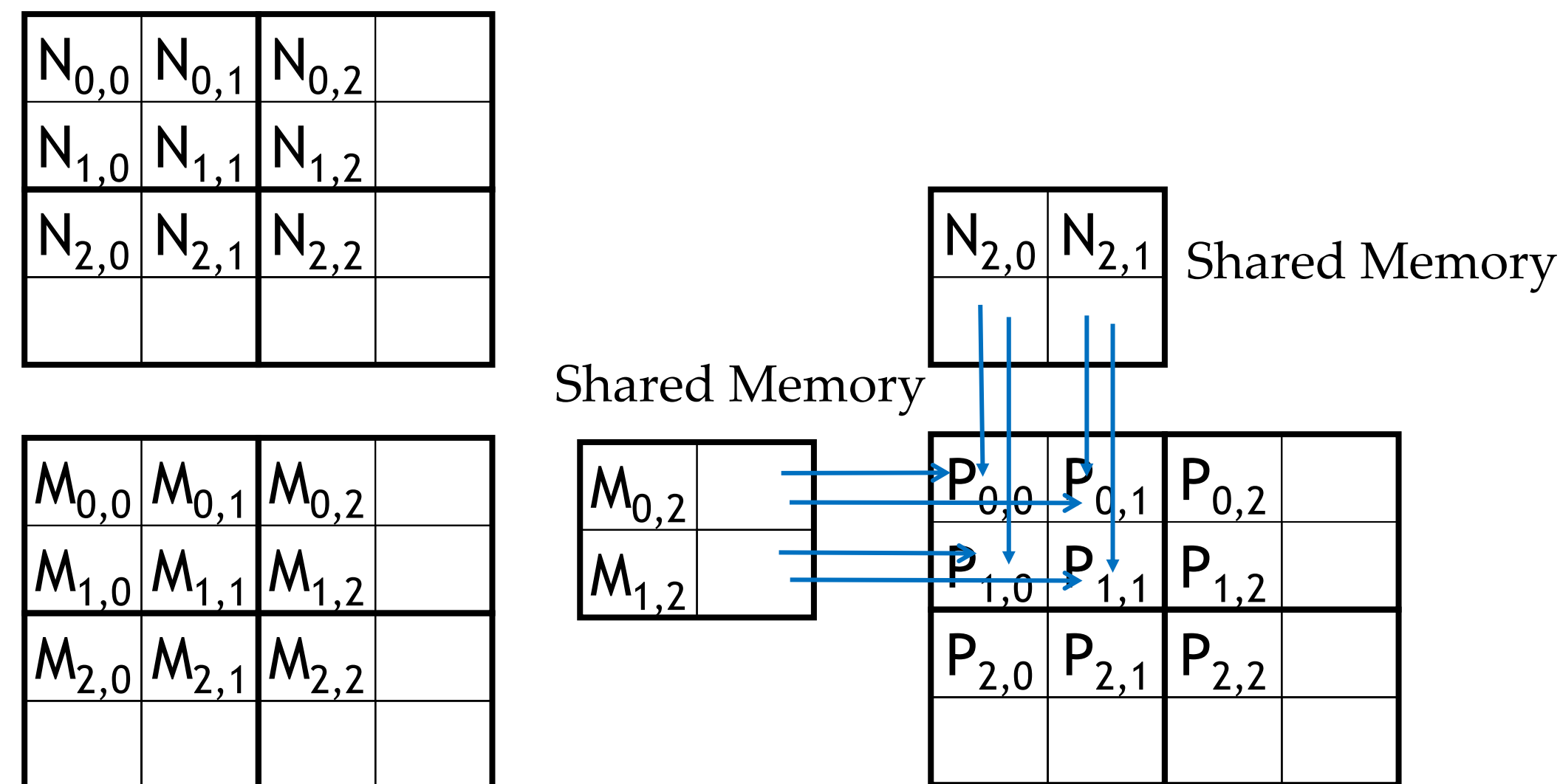
Phase 1 Loads for Block (0,0) for a 3x3 Example



Phase 1 Use for Block (0,0) (iteration 0)



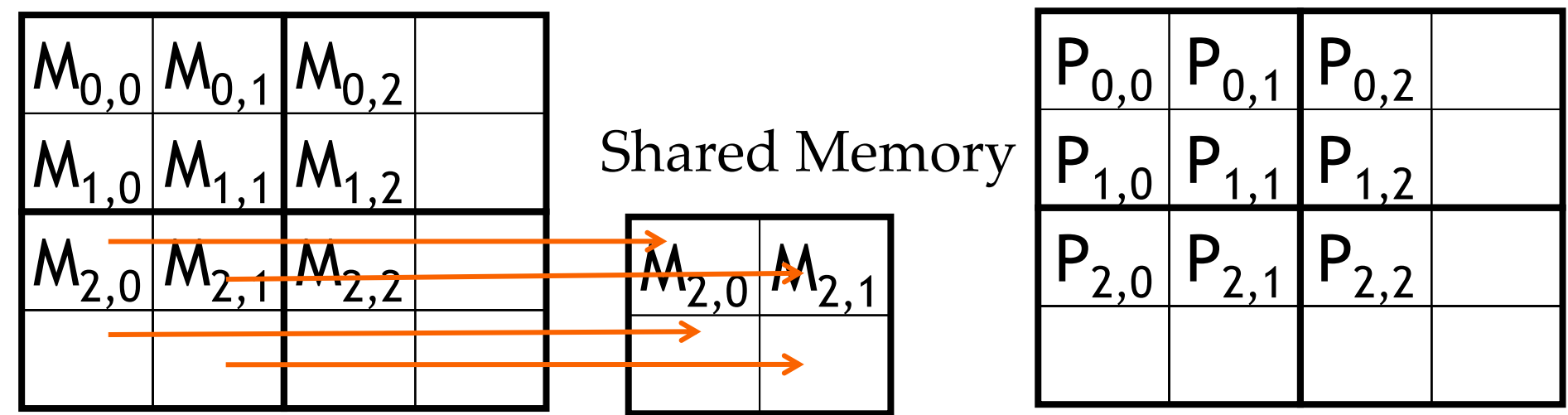
Phase 1 Use for Block (0,0) (iteration 1)



All Threads need special treatment.
None of them should introduce
invalidate contributions to their P
elements.

Phase 0 Loads for Block (1,1) for a 3x3 Example

Threads (0,1) and (1,1) need special treatment in loading N tile



Threads (1,0) and (1,1) need special treatment in loading M tile

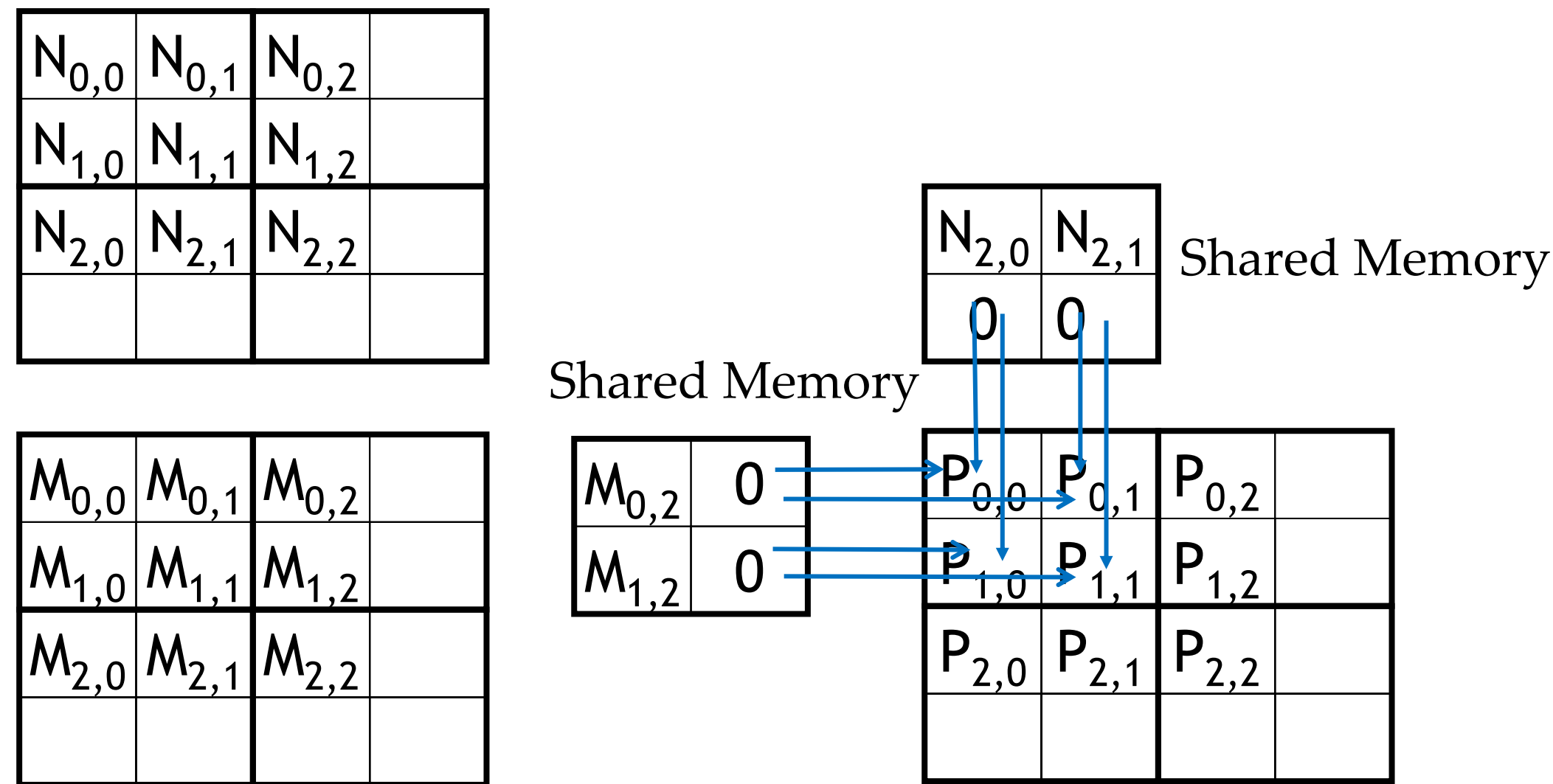
Major Cases in Toy Example

- Threads that do not calculate valid P elements but still need to participate in loading the input tiles
 - Phase 0 of Block(1,1), Thread(1,0), assigned to calculate non-existent $P[3,2]$ but need to participate in loading tile element $N[1,2]$
- Threads that calculate valid P elements may attempt to load non-existing input elements when loading input tiles
 - Phase 0 of Block(0,0), Thread(1,0), assigned to calculate valid $P[1,0]$ but attempts to load non-existing $N[3,0]$

A “Simple” Solution

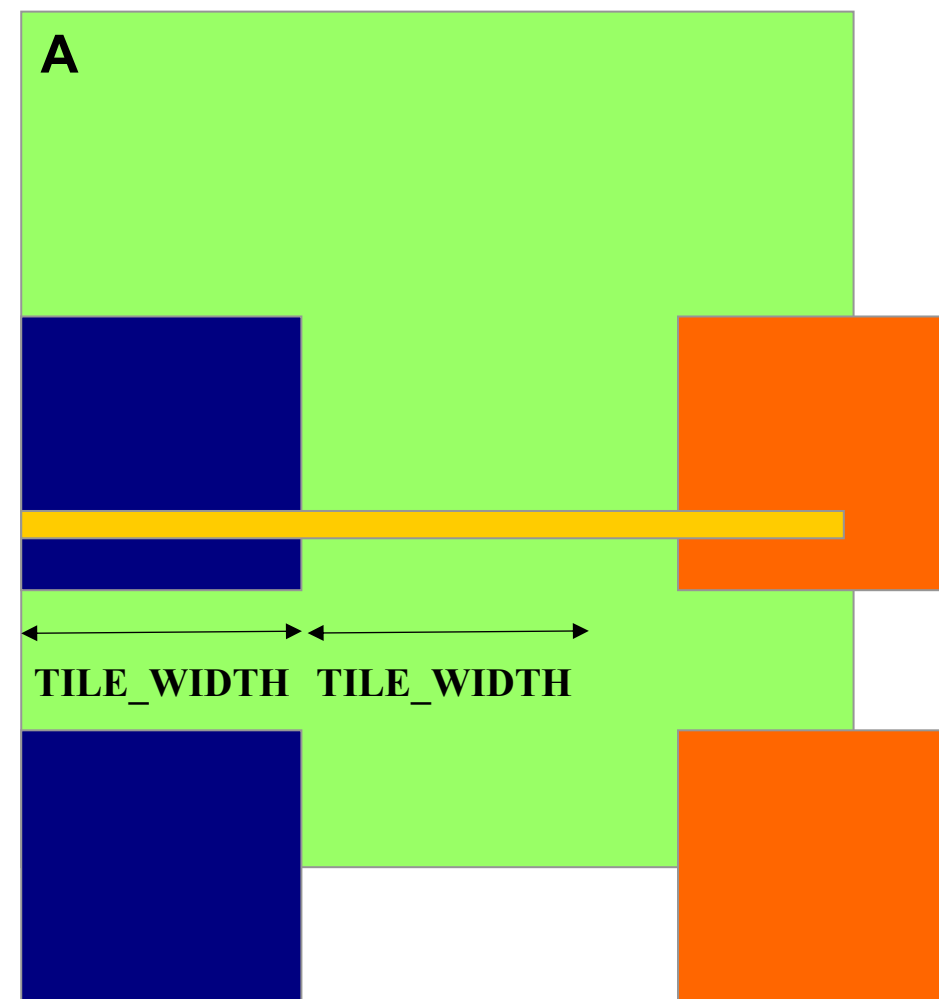
- When a thread is to load any input element, test if it is in the valid index range
 - If valid, proceed to load
 - Else, do not load, just write a 0
- Rationale: a 0 value will ensure that the multiply-add step does not affect the final value of the output element
- The condition tested for loading input elements is different from the test for calculating output P element
 - A thread that does not calculate valid P element can still participate in loading input tile elements

Phase 1 Use for Block (0,0) (iteration 1)



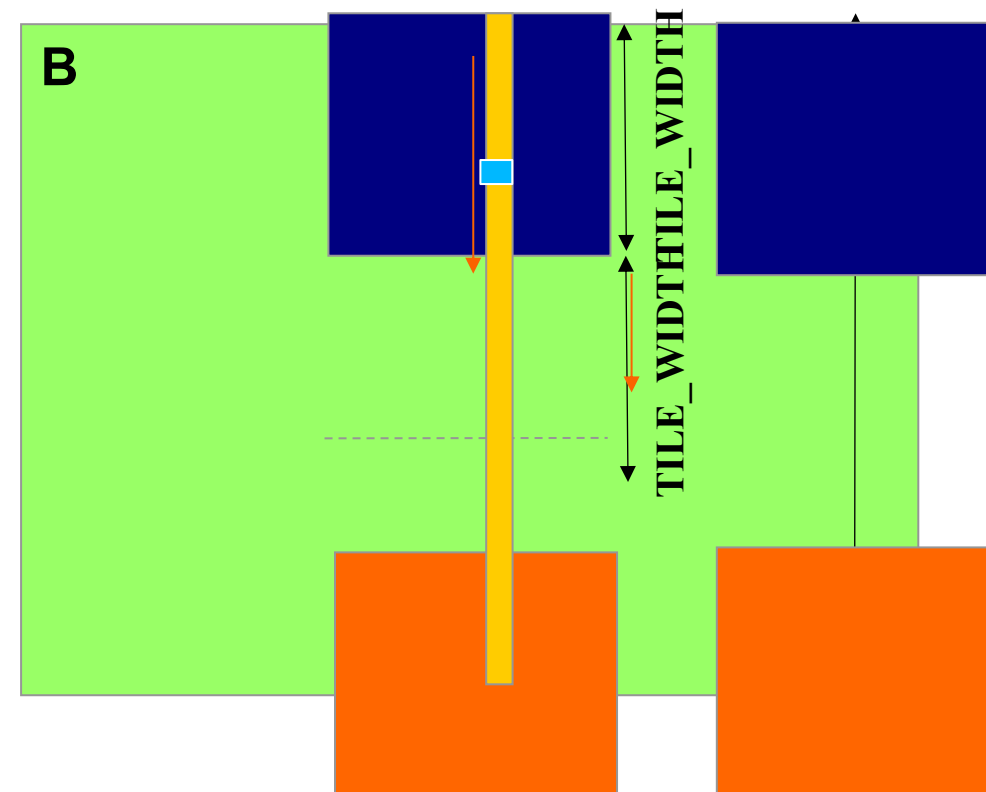
Boundary Condition for Input M Tile

- Each thread loads
 - $M[\text{Row}][p * \text{TILE_WIDTH} + tx]$
 - $M[\text{Row} * \text{Width} + p * \text{TILE_WIDTH} + tx]$
- Need to test
 - $(\text{Row} < \text{Width}) \ \&\& \ (p * \text{TILE_WIDTH} + tx < \text{Width})$
 - If true, load M element
 - Else , load 0



Boundary Condition for Input N Tile

- Each thread loads
 - $N[p \cdot \text{TILE_WIDTH} + ty][\text{Col}]$
 - $N[(p \cdot \text{TILE_WIDTH} + ty) \cdot \text{Width} + \text{Col}]$
- Need to test
 - $(p \cdot \text{TILE_WIDTH} + ty < \text{Width}) \ \&\& \ (\text{Col} < \text{Width})$
 - If true, load N element
 - Else , load 0



Loading Elements – with boundary check

```
– 8  for (int p = 0; p < (Width-1) / TILE_WIDTH + 1; ++p) {  
–  
– ++    if(Row < Width && p * TILE_WIDTH+tx < Width) {  
– 9      ds_M[ty][tx] = M[Row * Width + p * TILE_WIDTH + tx];  
– ++    } else {  
– ++      ds_M[ty][tx] = 0.0;  
– ++    }  
– ++    if (p*TILE_WIDTH+ty < Width && Col < Width) {  
– 10     ds_N[ty][tx] = N[(p*TILE_WIDTH + ty) * Width + Col];  
– ++    } else {  
– ++      ds_N[ty][tx] = 0.0;  
– ++    }  
– 11  __syncthreads();  
–
```

Inner Product – Before and After

```
– ++  if(Row < Width && Col < Width) {  
– 12      for (int i = 0; i < TILE_WIDTH; ++i)  
– 13          Pvalue += ds_M[ty][i] * ds_N[i][tx];  
–      }  
– 14  __syncthreads();  
– 15 } /* end of outer for loop */  
– ++  if (Row < Width && Col < Width)  
– 16      P[Row*Width + Col] = Pvalue;  
– } /* end of kernel */
```

Some Important Points

- For each thread the conditions are different for
 - Loading M element
 - Loading N element
 - Calculating and storing output elements
- The effect of control divergence should be small for large matrices

Handling General Rectangular Matrices

- In general, the matrix multiplication is defined in terms of rectangular matrices
 - A $j \times k$ M matrix multiplied with a $k \times l$ N matrix results in a $j \times l$ P matrix
- We have presented square matrix multiplication, a special case
- The kernel function needs to be generalized to handle general rectangular matrices
 - The Width argument is replaced by three arguments: j, k, l
 - When Width is used to refer to the height of M or height of P, replace it with j
 - When Width is used to refer to the width of M or height of N, replace it with k
 - When Width is used to refer to the width of N or width of P, replace it with l